

ConstructionSceneReady: Evidence-grounded compilation and task-scene validation for BIM-driven robotic assembly

Abstract

Robot-ready construction simulation requires more than converting building geometry into a simulation format. We ask which building information actually supports a robotic construction task in simulation, and how its absence, error, or imprecision changes task outcomes. ConstructionSceneReady compiles IFC source into task-activated multi-scale OpenUSD scenes and validates them against an explicit task-scene contract with evidence-bounded repair. On 105 fault-injection cases the contract recovers all 141 findings without false positives and repairs 97.1% to pristine equivalence; an agent constrained to the same whitelist preserves it, while an unconstrained agent produces incorrect-but-passing scenes in 22.7%. Paired humanoid episodes show that *wrong* symbolic and referential values are discretely fatal while a *missing* semantic label still degrades outcomes. The 12 mm reward criterion also masks anchor-relative interface errors: geometrically valid insertion collapses to 2.0% at a 10 mm hole-coordinate error while reward success stays $\sim 80\%$, a blind spot the contract closes with two independently verified rules.

Keywords: building information modeling, OpenUSD, construction robotics, humanoid robot, simulation validation, agentic repair

1. Introduction

Robotic construction is increasingly planned from digital building information, yet the model delivered by a designer and the world required by a robot remain fundamentally different artefacts. An Industry Foundation Classes (IFC) model describes intended components, spatial containment, materials, and design relationships. A robot simulator instead requires collision geometry,

mass properties, physical constraints, task targets, and an explicit account of which objects and geometric details are active. The difference is not resolved by changing file formats. It is a change from a representation of what should be built to a representation of what can be acted upon, under a particular construction state and by a particular embodiment.

The problem becomes more pronounced at building scale. Navigation through a floor or

room can be evaluated with simplified walls, openings, and obstacles, whereas placing a connection plate or inserting a pin depends on millimetre-scale interface geometry. Loading every building object at interaction-level fidelity is wasteful and can make collision checking and physics simulation unnecessarily expensive. Retaining only coarse geometry is equally problematic: it can conceal penetrations, reverse an insertion axis, or turn an infeasible assembly into an apparently valid one. A useful robot world must therefore change its working set and geometric fidelity with the task without losing identity or relationships across scales.

Previous studies have established important parts of this information chain. BIM-derived robot worlds and task-planning systems have been generated for painting, prefabricated assembly, and interior wall installation [1–3]. IFC semantics have also been linked to action-level robot control through the Robot Construction Action Node [4], while closed-loop digital twins have used as-built observations and human intervention to adjust BIM-driven construction workflows [5]. These studies show that building information can drive robot planning and simulation. They do not, however, establish a general criterion for deciding whether a composed construction scene is ready for a specified robot task before that task is executed.

In parallel, physics-ready asset generation has become considerably more automated. Xu et al. converted construction-equipment CAD data and mechanical specifications into articulated USD models, using a large language model to recover miss-

ing joint information [6]. OpenUSD now provides a strong technical basis for composing such assets through layers, references, and payloads [7]. Ren et al. have already shown that information delivery requirements can drive a specification-based IFC-to-USD pipeline while preserving life-cycle identifiers and model structure [8]. NVIDIA’s SimReady framework further organizes machine-checkable asset requirements into profiles, features, and rules [9]. This progress is essential, but an individually valid beam, tool, robot, or collider does not imply a valid assembly scene. The assets may refer to inconsistent coordinate frames, a task may address the wrong IFC object, the required work-zone payload may be absent, or the construction state may violate an action precondition. The current SimReady rules operate at asset and feature level; the official documentation notes that scene-level specifications and validators are not yet defined [10].

Figure 1 summarizes the resulting transition. Asset-oriented checks can establish that individual models contain valid geometry, collision shapes, or physical attributes, but the construction task is evaluated over a composition. Readiness therefore depends on closed cross-asset references, consistent spatial frames, the active work-zone payload, construction-state preconditions, and an interaction model whose fidelity matches the intended action.

Construction makes this scene-level gap particularly consequential. Temporary states such as pre-positioned, aligned, partially inserted, and temporarily fixed are not nor-

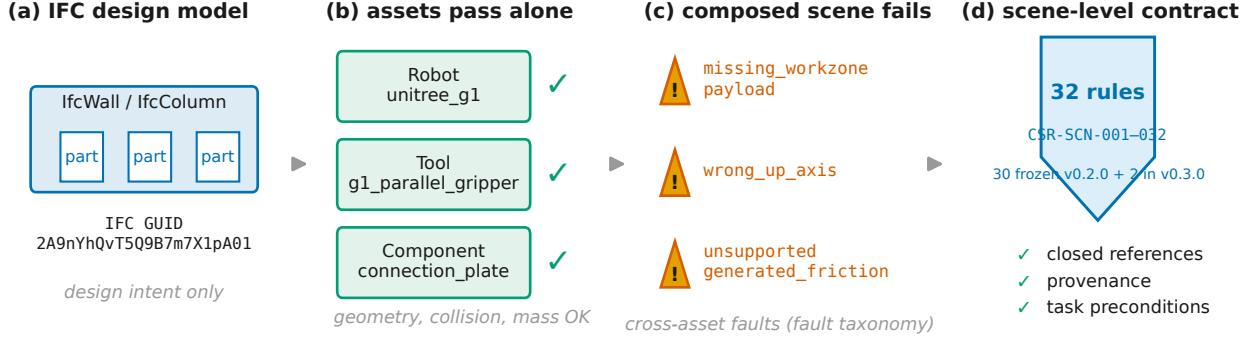


Figure 1: From individually valid digital assets to a task-ready construction scene. Scene-level failures emerge from composition, construction state, and task binding even when each asset passes its own checks.

mally represented in an as-designed model, although they determine whether the next robotic action is safe and meaningful. Physical properties may also be assembled from sources of unequal authority: geometry and identity may be explicit in IFC, mass may be calculated from volume and material density, friction may come from a published interval, and an assembly tolerance may be introduced as a task-specific assumption. If these values are stored without provenance, a simulator can produce a precise-looking result whose physical basis cannot be audited. A language-model agent compounds the problem when it is allowed to invent or modify engineering parameters without a deterministic acceptance test.

Humanoid robots provide a demanding but useful setting in which to examine these issues. Human-oriented buildings contain doors, thresholds, narrow work zones, tools, and operating heights that motivate legged mobility and whole-body manipulation. This does not make a humanoid uni-

versally preferable to a mobile manipulator; wheeled systems retain clear advantages in payload, stability, and operation on regular floors. Rather, a humanoid task exposes the need to connect building-scale circulation, body placement, bimanual reach, and local contact geometry in one composed scene. The present work therefore uses light-steel assembly by a Unitree G1-class humanoid as the evaluation context, while keeping the proposed scene representation independent of a particular controller. A recent construction-humanoid digital-twin study has already demonstrated the conversion of BIM geometry and semantic information into a MuJoCo scene for Unitree G1 task decomposition and simulation [11]. The question pursued here is consequently not whether BIM and a humanoid can coexist in a simulator, but whether the resulting multi-asset scene can be compiled, audited, repaired, and related quantitatively to task outcome.

This paper develops Construction-SceneReady around the proposition that

construction scene readiness should be compiled and tested explicitly, rather than discovered only through failed simulation runs. The central research question is which building information supports a robotic assembly task in simulation, and how its absence, error, or imprecision changes the outcome. IFC4.3 entities are first translated into a construction scene intermediate representation that retains component identity, work zones, assembly interfaces, construction states, tasks, and property provenance. The representation is then compiled into a global navigation layer, one or more work-zone payloads, and task-specific interaction layers. This organization permits OpenUSD to load detailed geometry only where the active task requires it, while references preserve the connection to the same engineering objects across layers.

A deterministic contract is evaluated over the composed scene rather than over isolated assets. Its rules cover coordinate and unit consistency, reference closure, task-to-interface binding, state preconditions, work-zone activation, and the availability and provenance of critical physical parameters. When a rule fails, an agent receives a structured diagnostic and may select only from a whitelist of repair operations, such as restoring an IFC–USD mapping, recomputing inertia, or regenerating an allowed payload. The validator, rather than the agent, decides whether the revised scene is acceptable. Parameters whose authority or risk exceeds a defined threshold are escalated for human review.

The evaluation is built around controlled

failure rather than a single successful demonstration. Parameterized IFC4.3 fixtures generate correct scene descriptions and machine-readable ground truth for connection-plate positioning, pin insertion, and sequential assembly. Known semantic, spatial, physical, interface, state, and task defects are then injected to compare generic asset checks, fixed-rule conversion, unconstrained agentic repair, and validator-grounded repair. CPU experiments measure detection, repair reliability, provenance coverage, and the resource effect of task-activated layers. The central experiment then measures the downstream consequence of each information family on a humanoid bolt-insertion task executed by a frozen reinforcement-learning policy, and uses the measured consequences to recalibrate the contract itself. In this way, the paper links an information-model claim—that a scene satisfies an explicit construction contract—to an operational claim—that the composed world supports the intended robotic task.

2. Related work

2.1. Construction robotics and the role of building information

Construction robotics has long been framed as an information problem as much as a machine problem. Early surveys anticipated that sensing and product models would be needed to compensate for site variability [12–14]. Later reviews identified persistent barriers in localization, work preparation, safety, interoperability, and adaptation

to unstructured conditions [15–17]. The diversity of the field is relevant to scene modelling: collective fabrication, on-site mobile systems, prefabrication, and human-robot collaboration impose different requirements on the digital representation used at runtime [18–20]. Consequently, “robot-ready” cannot denote a single geometric level of detail or one fixed set of asset attributes.

Reviews focused specifically on BIM-robot interaction show a progression from one-way export towards lifecycle information flow and closed-loop exchange [21–24]. Across this literature, BIM supplies geometry, object identity, material and spatial semantics, schedules, and task constraints. The receiving robotic system commonly creates its own reduced representation for a particular purpose. A navigation map, an assembly graph, an SDF world, and an articulated physics asset are all robot-relevant, but they make different claims about what information is complete and valid. This fragmentation motivates the present emphasis on an explicit scene construct rather than another destination format.

For robotic assembly, Dawod and Hanna used BIM geometry and design context to support object recognition, grasp search, and motion planning for discrete structures [25]. Ding et al. connected image-based reconstruction to BIM task-level planning for brick assembly [26]. Wong Chong et al. automated the generation of robot simulation input for timber-frame assembly [27], while Gao et al. combined BIM-based task and motion planning for lightweight emergency structures [28]. These studies established that building objects can be transformed into ex-

ecutable task information; they did not seek to define a common acceptance test for the composed simulator stage.

Kim et al. developed a BIM-integrated planning and simulation system in which building information was converted to an SDF world and used to configure robot painting tasks [1]. Zhu et al. subsequently proposed an open, component-oriented pipeline that split IFC building elements, preserved their identity, converted them into SDF smart components, and executed a light-steel assembly sequence in ROS/Gazebo [2]. This work established automated IFC-to-simulator conversion as feasible, but assembly interface validity and cross-component physical consistency remained outside its scope.

Research then moved closer to action-level construction information. The RCAN framework linked IFC-derived data to ordered construction actions and ROS execution, enabling changes in component-oriented plans to be propagated to a mobile manipulator [4]. Oyediran et al. integrated 4D BIM, a construction-robot knowledge base, high-level task planning, and action-level simulation for indoor wall-frame installation [3]. These systems enrich BIM with task knowledge and make robot operations more context-aware. Their principal concern is the creation and flow of planning information, rather than a formal test of whether all assets, references, intermediate states, and physical assumptions form a valid task scene.

Semantic world modelling provides a second route from BIM to robot execution. Kim and Peavy represented building elements,

spaces, and robot-relevant relationships in a semantic building world [29]. For mobile robots, IFC has been converted into maps, waypoints, and schedule-aware Gazebo environments [30, 31]. Knowledge-driven inspection and progress-monitoring systems likewise use BIM to select targets and interpret observations [32, 33]. Ontologies have been proposed to regularize the vocabulary for navigation and inspection [34], and semantic BIM attributes have been used to alter path cost and clearance in safety-aware planning [35]. This line of work demonstrates the operational value of semantics, but its main unit of analysis is a map, waypoint, or knowledge base. Assembly readiness additionally requires valid contact interfaces, physical parameters, object states, and task success conditions.

2.2. Physics-ready assets and multi-scale scene composition

Robotic simulation depends on asset properties that BIM models rarely provide in executable form. Xu et al. addressed this issue for construction equipment by fusing CAD geometry, equipment templates, and technical specifications into articulated USD models [6]. Their method generated hierarchy, joints, drives, materials, and collision representations for a wheel loader, and used a large language model to extract unspecified mechanics. The study substantially reduced modelling time and is a close precedent for automated physics-ready generation. Its stated scope was equipment modelling rather than environment modelling or equipment-environment interaction. It there-

fore does not resolve the composition of building components, temporary states, task interfaces, and robot capabilities into one assembly scene.

General-purpose robotics simulators make the representational gap visible in different ways. MuJoCo emphasizes efficient rigid-body dynamics and contact-rich model-based control [36]; GPU-native simulation environments such as Isaac Gym make it practical to execute many robot worlds in parallel [37]. These runtime capabilities do not establish that a source model was compiled correctly. Domain randomization can expose policies to variations in visual or physical parameters [38], but it presupposes a valid nominal model and defensible parameter ranges. It is therefore distinct from detecting a reversed coordinate frame, a missing assembly mate, or a physical value with no authoritative source.

OpenUSD is well suited to scene composition because its layers, references, variants, and payloads separate asset definitions from scene opinions and permit data to be loaded on demand [7]. This is important for construction: a building shell may support navigation at coarse resolution while an active connection requires detailed collision and contact geometry. Ren et al. developed a specification-driven IFC-to-USD pipeline in which information delivery requirements act as a scene contract for selecting and mapping identifiers, hierarchy, geometry, materials, and attributes [8]. Their study establishes both lifecycle-oriented semantic preservation and configurable IFC-to-USD delivery; neither can be claimed as new here. What remains unresolved for robot construction is

how a current task should activate geometric scales, physical interfaces, temporary states, and cross-asset acceptance conditions. If cOpenShell provides access to IFC4.3 objects, property sets, spatial containment, and geometry [39, 40], but a domain-specific compilation policy is still required to turn that information into an auditable task scene.

SimReady complements USD composition with machine-readable capabilities, features, profiles, and validation rules [9]. Current profiles are valuable for checking individual assets, including rigid-body, collider, material, hierarchy, and robot-articulation requirements. An asset-level result cannot establish that a particular robot, tool, work zone, component, and target interface have been composed consistently. The official FAQ identifies scene-level specifications and validators as a limitation of the current framework [10]. ConstructionSceneReady therefore builds on, rather than replaces, asset validation by adding relations that become meaningful only after a construction task scene is composed.

Scene generation for embodied AI is advancing in parallel and narrows the claim space further. SimRecon reconstructs compositional simulator scenes from video through perception, object generation, and scene-graph-guided assembly [41]. EmbodiedGen V2 generates editable, task-driven worlds with affordances and cross-simulator deployment, and evaluates whether the worlds support downstream policy learning [42]. Agentic Real2Sim recovers geometry, physical parameters, robot interaction, and object state from recorded episodes using vision-language

agents [43]. These systems show that agentic production of runnable, physically enriched scenes is no longer an unoccupied research problem. Their source of authority is visual observation or generative world specification, whereas a construction scene must also remain accountable to IFC identity, engineering interfaces, process state, and explicitly bounded parameter sources. This difference motivates an auditable compiler and fault benchmark rather than another general-purpose scene generator.

The use of a humanoid evaluation platform makes this cross-scale requirement especially visible. Whole-body benchmarks show that locomotion and manipulation outcomes depend on coupling among support, reach, contact, and object dynamics [44]. In a construction scene, the coupling extends across building circulation, body placement, work-zone occupancy, and millimetre-scale assembly geometry. A compiler must support coarse global context and detailed active interfaces without representing every building object at maximum fidelity. This is a scene-composition requirement, not a claim that humanoids are universally preferable to wheeled mobile manipulators. Nor is BIM-to-humanoid simulation itself new: Ye et al. convert BIM geometry and semantic information to a MuJoCo scene and demonstrate task decomposition with Unitree G1 [11]. The present use of G1 is therefore an evaluation stress test for task-scene readiness rather than a standalone contribution.

2.3. Closed-loop and robot-ready construction twins

445 Digital-twin research has addressed the discrepancy between design models and changing construction sites. Boje et al. argued for semantically structured construction twins that connect heterogeneous representations and lifecycle processes [45]. Sacks et al. placed the twin within a closed-loop production system in which observations and decisions continually update construction control [46]. These formulations show that a design model is only one view of a changing physical system. They do not, however, prescribe a simulator-specific readiness contract for robotic task.

460 Wang et al. proposed a closed-loop BIM-driven workflow in which a human could revise task sequences, target positions, and motion plans when drywall installation encountered as-designed/as-built differences [5]. Tuomisto et al. used BIM to initialize LiDAR SLAM and next-best-view planning for quadruped inspection, while allowing the map to distinguish designed structures, missing elements, and dynamic obstacles [47]. These approaches make robot operation robust to observed site conditions, but they validate the world through runtime sensing and interaction rather than through pre-runtime analysis of a composed simulation scene.

475 The BIM2RDT work of Akhavian et al. further integrates BIM geometry, robot observations, IoT data, semantic registration, and safety-aware navigation [48]. It represents an important step toward dynamic, robot-ready construction twins and LLM-orchestrated workflows. Its principal tech-

nical contributions concern point-cloud registration, perception, monitoring, and navigation constraints. The present work addresses a complementary stage: compiling an auditable physical scene before simulation, then determining whether its cross-asset relationships and construction-task assumptions are internally ready for execution.

Engineering analysis can also serve as an acceptance gate. The integration of BIM, finite-element analysis, and ROS has shown how temporary structural stability can modify a robot assembly plan [49]. BIM-semantic-web integration can supplement an observed component with design relationships and requirements [50]. These studies demonstrate that readiness is conditional on the current state and action, not an intrinsic label attached to an asset. ConstructionSceneReady adopts that conditional view while limiting its present physical scope to rigid components and simplified contact interfaces.

2.4. Language models, agents, and engineering assurance

Language models have begun to support construction-robot planning and engineering-model generation. RoboGPT used ChatGPT to generate and revise assembly sequences, demonstrating that natural-language reasoning can adapt a robot plan across construction cases [51]. Xu et al. used an LLM more narrowly to extract equipment-joint parameters from technical documents [6]; BIM2RDT uses language-model reasoning to assign semantic priors and orchestrate parts of a robot-ready twin pipeline [48]. These studies

show the value of models that connect heterogeneous descriptions and select procedural steps.

The wider robotics literature offers several patterns for grounding language in executable behavior. SayCan combines language-model preferences with learned affordance values [52]; Inner Monologue uses environment feedback to revise embodied plans [53]. Code as Policies and Program Prompt constrain generated output through executable program structure or situated action specifications [54, 55]. Multimodal embodied models such as PaLM-E and vision-language-action models such as RT-2 move perception and action closer to the language model itself [56, 57]. Multi-robot planning frameworks further decompose tasks, allocate capabilities, and produce programmatic plans [58]. These methods strengthen task generation, but none of them makes a language model an authoritative source for an engineering mass, tolerance, coordinate frame, or IFC identity.

A plausible action sequence is not necessarily executable, and a plausible numerical parameter is not necessarily authoritative. In ConstructionSceneReady, the agent is not treated as a validator or a source of ground truth. It receives a typed error report, chooses from bounded repair tools, records the evidence for each mutation, and returns the scene to deterministic rules. This pattern differs from direct language-to-scene generation because acceptance remains independent of the model that proposed the repair. It also makes fixed-rule and agentic methods comparable on identical injected faults.

The separation of proposal from acceptance is consistent with hybrid planning in which a language model supplies a rough plan and a formal answer-set planner establishes executability [59]. It also aligns with fault-oriented robot assurance: injected faults can determine whether a digital representation and its deployment process expose unsafe assumptions before hardware execution [60]. In the present benchmark, the object under test is the construction scene. The agent may select a repair, but typed rules, source authority, and task-level checks determine whether that repair is accepted.

2.5. Synthesis of the research gap

Table 1 compares the closest methodological precedents. The reviewed literature establishes IFC-to-simulator conversion, BIM-derived robot tasks, semantic maps, specification-driven IFC-to-USD delivery, physics-ready USD asset generation, agentic simulation-world generation, BIM-to-humanoid simulation, closed-loop twin updating, semantic perception, and language-model-assisted planning. These capabilities are necessary parts of a robot-ready workflow, but they operate on different units of analysis. Conversion studies focus on delivered building information, physics-ready studies on an equipment asset, generative-world studies on executable scenes, planning studies on action information, and digital-twin studies on runtime alignment with a site.

The literature search did not identify a study that jointly evaluates a construction-task scene contract, task-activated multi-scale USD composition, physical-property

provenance, deterministic scene-level fault de-
 tection, validator-grounded repair, and the
 downstream outcome of a humanoid assem-
 595 bly task. This is deliberately narrower than
 a claim of first IFC–robot interoperability or
 first USD validation; both are established ar-
 eas. The unresolved transition is from indi-
 vidually valid digital assets to an auditable,
 600 composed, task-specific construction world.

Two buildingSMART instruments deserve
 explicit positioning. The Information Deliv-
 ery Specification (IDS) [61] provides machine-
 checkable information requirements for IFC
 605 deliveries, and the buildingSMART Data Dic-
 tionary (bSDD) [62] standardizes the vocabu-
 lary those requirements reference. IDS is
 more expressive than a per-attribute check:
 its facets include a `PartOf` facet that con-
 610 strains how an object belongs to aggrega-
 tions, groups, spatial containers, and nests,
 so relationship requirements are in scope for
 IDS. The present contract is not a claim
 to introduce relation checking for the first
 615 time; its distinct subject is the *robotic task*
scene: that a task target must resolve to
 the component owning the bound interface,
 that an active work zone’s payload must stay
 inside the scene package, that a construc-
 620 tion state must gate the next action, and
 that a critical physical value must carry an
 authoritative provenance record. These are
 cross-asset bindings, final-composition safety,
 process-state, and external-evidence consis-
 625 tency requirements, which are outside what
 an IFC delivery—even an IDS-conformant
 one—is asked to certify. The contract is com-
 plementary by design: bSDD-style dictionar-
 ies supply standardized semantics, IDS-style

facets supply entity and aggregation require-
 ments, and the scene contract adds cross-
 reference closure, task binding, composition
 safety, and evidence authority at the level of
 the composed scene. Section 7 further shows
 empirically that even a fully satisfied contract
 leaves sub-tolerance geometric fidelity invis-
 ible, which motivates the candidate rules reg-
 istered there.

Table 1: Capabilities of closely related BIM–robotics and physics-ready modelling studies. A filled cell indicates that the capability was a substantive part of the reported method or evaluation.

Study	BIM/IFC	Robot task	Physical asset	Multi-scale	Scene rules	Provenance	Agent repair	Runtime outcome
Kim et al. [1]	✓	✓	partial	–	–	–	–	✓
Wong Chong et al. [27]	✓	✓	partial	–	–	–	–	✓
Zhu et al. [2]	✓	✓	partial	–	–	–	–	✓
Oyediran et al. [3]	✓	✓	partial	–	–	–	–	✓
Xu et al. [6]	–	–	✓	–	asset	partial	partial	✓
Ren et al. [8]	✓	–	partial	scene	specification	✓	–	–
Wang et al. [5]	✓	✓	partial	–	runtime	–	human	✓
Gopee et al. [31]	✓	navigation	partial	map	runtime	–	–	✓
Akhavian et al. [48]	✓	navigation	partial	–	runtime	partial	partial	✓
Ye et al. [11]	✓	✓	partial	scene	–	–	–	✓
Wang et al. [42]	–	✓	✓	✓	partial	partial	generation	✓
ConstructionSceneReady	✓	✓	✓	✓	scene	✓	✓	✓

3. Problem definition

Let I be an IFC model, K a set of construction-domain rules, R a robot description, and T a task specification. The compiler produces an OpenUSD scene S and a provenance ledger P :

$$C(I, K, R, T) \rightarrow (S, P). \quad (1)$$

The scene is composed from three fidelity layers:

$$S = S_{\text{global}} \oplus S_{\text{workzone}} \oplus S_{\text{interaction}}, \quad (2)$$

where $\oplus$ denotes OpenUSD composition. The active task determines which work-zone and interaction payloads are loaded.

A construction scene contract is a versioned set of requirements $\mathcal{Q} = \{q_1, \dots, q_n\}$. A deterministic validator returns:

$$V(S, T, \mathcal{Q}) \rightarrow E, \quad (3)$$

where E is a structured set of violations containing a rule identifier, severity, scene path, evidence, and allowed repair tools.

For every critical generated property p_i , the compiler stores:

$$p_i = (v_i, s_i, c_i, u_i), \quad (4)$$

where v_i is the value, s_i its source, $c_i \in [0, 1]$ a confidence score, and u_i an uncertainty description or interval.

Repair is then a constrained selection, not a probability maximization, with two distinct acceptance levels. Each violation $e \in E$ carries a small allowed-tool set $\mathcal{A}(e) \subseteq \mathcal{A}$ registered by its rule. Writing E_0 , E_1 for the violation sets before and after a candidate fix $a \in \mathcal{A}(e)$, and $B(\cdot)$ for the blocking (critical) members, a single repair step is *submitted* only if it resolves its target and adds no new blocking violation, so a no-op or a mutation that merely hides the original violation cannot be accepted:

$$\begin{aligned} \text{submit}(a, e) \iff & e \notin E_1 \wedge \\ & [B(E_1) \setminus B(E_0) = \emptyset] \wedge \\ & \text{EvidenceOK}(a, P). \end{aligned} \quad (5)$$

This is intentionally local: in a compound-fault scene, fixing e_1 while an unrelated e_2

is still open does not block the step. The repaired scene is then *accepted* only when no blocking violation remains:

$$\text{accept}(S') \iff B(E_{\text{final}}) = \emptyset, \quad (6)$$

where a violation’s identity is its rule and entity, not its diagnostic text. Unsupported or safety-critical mutations are escalated to human review. The whitelisted tool registry and this two-level re-validation gate are the implemented mechanism; no probability is estimated and no such optimization is solved at runtime.

Table 2 organizes the contract families by the robot function they serve, the task phase in which they act, and the measured consequence of their absence or error. Three levels are distinguished throughout: whether a building-information item can be *expressed* in the scene, whether the executor actually *uses* it, and whether its corruption measurably *affects* the task outcome.

Provenance of the task information.. Table 3 states, for each field the executor consumes, where the value comes from and who is responsible for it. Four source classes are distinguished: *IFC-native* values already present in the delivery, *derived* values computed from IFC geometry or material, *task annotation* supplied with the robot–task specification (the CSR property-set layer), and *assumed* defaults that must be escalated rather than trusted silently. The mapping makes explicit that the pipeline does not pretend that every required quantity is IFC-native: interface coordinates are extracted from the IFC hole feature and then certified against an evidence

record; clearance and state are annotation responsibilities; mass is derived with an explicit uncertainty; and an unannotated field is by construction assigned to the escalation path, not to a guessed default (Section 7).

Table 2: Task-information mapping for the bolt-insertion assembly task. For each building-information family: the robot function it serves, the task phase in which it acts, and the measured consequence of its absence or error (Section 7.3). “Scaffolded” marks families whose dynamical channel is removed by the virtual-grasp executor; their task consequence is explicitly *not* claimed by this study.

Information family	Robot function served	Task phase	Measured consequence of defect
Semantic identity & task reference	Select which member, interface and hole to act on	target selection	<i>wrong</i> value discretely fatal (0%); <i>missing</i> label degrades (12.0% reward, 1.0% geo.); no perceptual compensation for a wrong target
Member pose (absolute)	Coarse approach and reach planning	transport	largely compensated: reward -4.5 pp (significant on seed 42, $p=0.012$), geo. -2.0 pp (n.s.)
Interface coordinates relative to member	Fine alignment at the hole mouth	align, insert	reward-tolerance partly masks it (-1.0 pp at 2–5 mm, n.s.) but geometric-valid degrades strongly ($-12.5/-14.0$ pp at 2/5 mm, -41.5 pp at 10 mm)
Interface clearance & tolerance	Physical insertability of the fastener	insert	sensitive continuous feature; -54.5 pp reward at 0.75 mm clearance
Construction state & sequence	Gate whether the action is legal now	precondition check	discretely fatal (0%)
Physical properties (mass, friction)	Force-level contact and stability	contact stages	<i>scaffolded</i> : no dynamical channel in the transport-and-insert executor (bit-identical outcomes); not ranked

Table 3: Provenance of task information: source class, extraction, runtime use, and repair responsibility. IFC-native = present in the delivery; derived = computed from IFC geometry/material; task annotation = CSR property-set/robot-task specification; assumed = default that must be escalated, never silently trusted.

Field		Source class	Extraction	Runtime use		Repair source
Component identity / IFC GUID	iden-	IFC-native	preserved through compilation	cross-reference	closure	IFC source
Member pose (absolute)		IFC-native	<code>IfcLocalPlacement</code> → USD transform	transport pose	target	IFC source
Interface coordinates (hole, member-relative)	coor-	derived	IFC hole feature → evidence record	fine alignment	at hole mouth	evidence record
Interface clearance / tolerance	clearance	task annotation	CSR property set → evidence record + provenance	geometric	insertability gate	evidence record
Construction state / sequence	state	task annotation	task state machine	precondition	gate	task file
Mass / inertia		derived	IFC volume × material density (+uncertainty)	physics	stability	derived value
Work zone / payload	zone / pay-	task annotation	CSR property set	multi-scale	activation	task file
Robot capability / preconditions	capability /	task annotation	robot-task specification	executor	configuration	task file
Unannotated required field	re-	<i>assumed</i>	none (absent)	none → escalate		human review

4. Methodology

including `located_in`, `has_interface`, `mates_with`, `performed_by`, `uses_tool`, `targets`, and `can_transition_to`. It is
715 deliberately simulator-neutral so that IFC extraction, graph-integrity checks, USD authoring, scene validation, and runtime adapters remain separately testable.

Figure 2 shows how these modules form one
720 auditable compilation pipeline. Data artefacts are identified on the arrows so that implementation outputs, validation evidence, and runtime measurements can be traced back to their source.

725 The typed entities and relations are specified in Figure 3. The figure also exposes provenance as part of the representation rather than as an external note attached after simulation.

4.1. Construction Embodied World Graph

730 4.2. Task-activated multi-scale compilation

The compiler maps building circulation and architectural boundaries to a coarse global layer. It emits each work zone as an independently loadable payload and reserves
735 interface-level geometry, tolerances, contact properties, and temporary constraints for the active interaction layer. IFC GUIDs remain stable across all layers.

Figure 4 illustrates the OpenUSD composition.
740 Its purpose is *working-set management*, not runtime acceleration: only the global layer and the active interaction layer are composed, so the loaded prim set tracks the active task rather than the whole model (Section 7 reports the measured working-set reduction; load time and peak memory are small and comparable across modes, so we do not claim a runtime-performance advantage).

705 The intermediate representation is implemented as a Construction Embodied World Graph (CEWG). Its typed nodes contain components, assembly interfaces, construction states, robots, tools, tasks, work zones,
710 robot capabilities, and provenance records. Edges make cross-entity claims explicit,

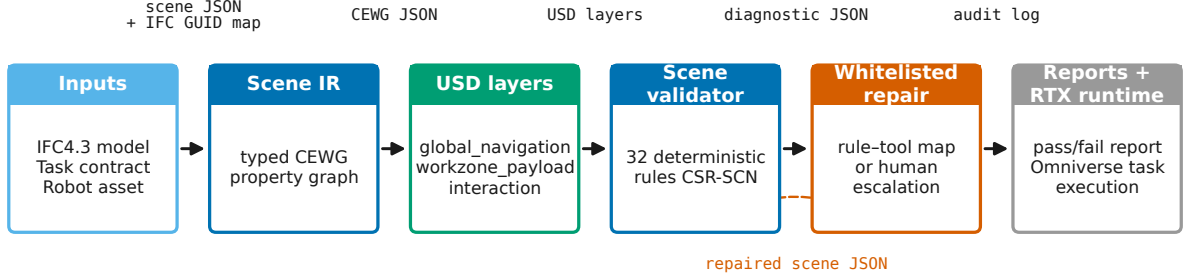


Figure 2: Architecture of ConstructionSceneReady. Deterministic compilation and validation remain separate from agentic diagnosis and from the downstream simulation runtime.

4.3. Construction scene readiness contract

Rules are grouped into identity and reference closure, coordinate and unit consistency, interface and tolerance completeness, state-transition validity, robot capability and task preconditions, physical stability, provenance, and multi-scale fidelity. Each rule is deterministic and emits a machine-readable failure.

Table 4 freezes the contract categories and the evidence required from the implementation. The full list of versioned rules and unit tests will be released as supplementary material rather than compressed into the main text.

4.4. Validator-grounded repair

The repair agent receives no free-form authority over the scene. It may inspect evidence, select a permitted tool, propose bounded parameters, execute the tool, and request re-validation. Unsupported or high-risk changes are escalated.

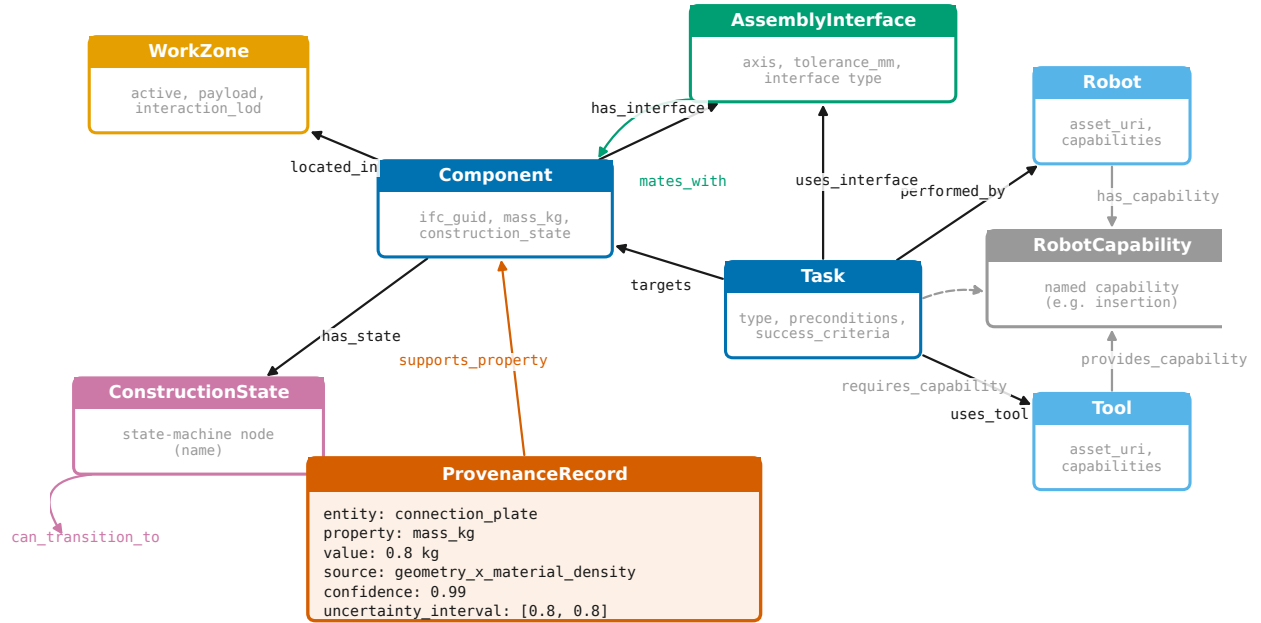


Figure 3: Construction Embodied World Graph and property-level provenance. The same engineering identity connects source IFC entities, composed USD prims, task targets, and validation evidence.

OpenUSD composition tree — pin_insertion (task_activated mode)

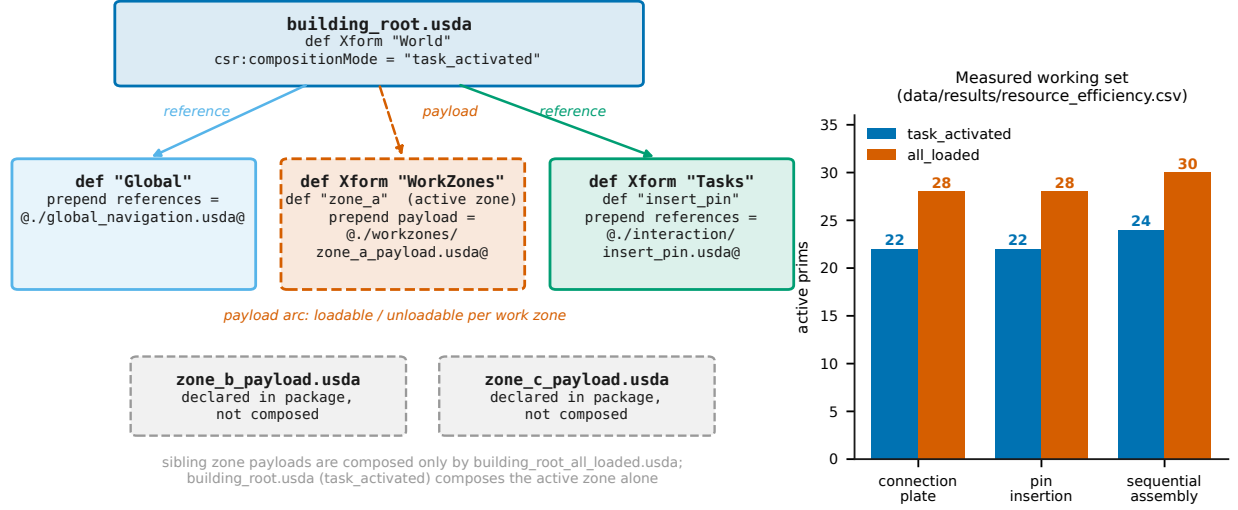
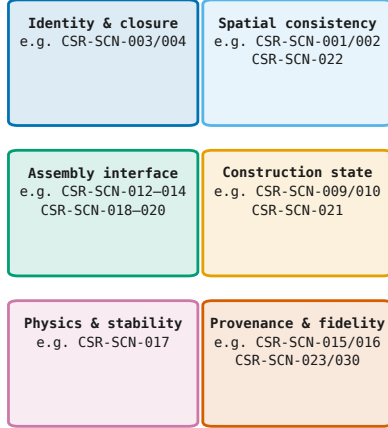


Figure 4: Task-activated multi-scale scene composition. Fine contact geometry and temporary constraints are loaded only for the active interaction while engineering identity is preserved across layers.

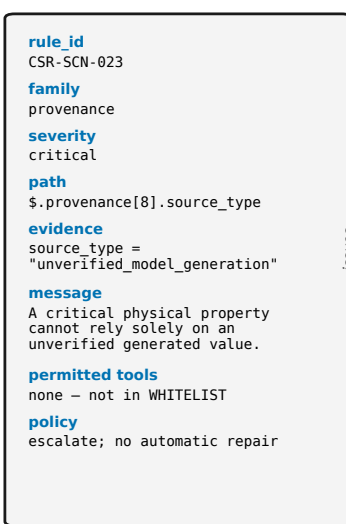
Contract families

(docs/scene-readiness-rules.csv)



30 deterministic rules (v0.2.0 baseline;
+2 in v0.3.0); every violation emits a
typed, machine-readable diagnostic

Diagnostic record (true validator output)



Repair state machine

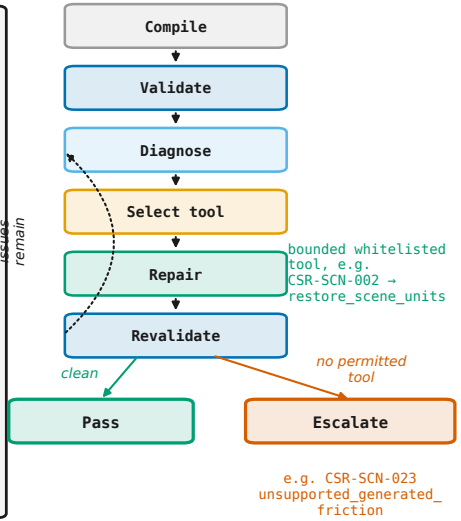


Figure 5: Validator-grounded repair. The agent interprets structured evidence and selects a bounded operation, whereas deterministic rules decide acceptance and high-risk or unsupported mutations are escalated.

Table 4: Construction-scene readiness contract (v0.3.0). The frozen v0.2.0 set carries 30 rules (CSR-SCN-001–030); v0.3.0 adds CSR-SCN-031 and CSR-SCN-032 for evidence-grounded interface fidelity, 32 rules in total. The versioned registry is exported at `docs/scene-readiness-rules.csv`, and per-scenario coverage is reported in the supplementary material.

Contract category	Required scene evidence	Deterministic check	Severity	Consequence if violated
Identity and closure	IFC GUID, USD prim path, closed task and asset references	Uniqueness and reference resolution	Critical	Wrong or unresolved task object
Spatial consistency	Units, up axis, frames, transform chain, work-zone membership	Transform normalization and bounds	Critical	Unreachable or misplaced interaction
Assembly interface	Paired interface, insertion axis, tolerance, grasp and support regions	Geometric and relational constraints	Critical	False alignment or infeasible insertion
Construction state	Current state, legal predecessor, action precondition, completion criterion	State-machine validation	Critical	Unsafe or meaningless action
Physics and stability	Mass, inertia, collision, contact property, temporary support	Physical admissibility and support tests	Critical or major	Unstable or non-physical simulation
Provenance and fidelity	Property source, confidence, uncertainty, activated payload and level of detail	Authority and task-fidelity policy	Major	Unauditable result or hidden interaction error

5. System implementation

The CPU-first implementation uses Python for the intermediate representation, validation, fault injection, and experiment orchestration. IfcOpenShell is used for IFC extraction [40]; the OpenUSD Python API authors composable scene layers [7]. Runtime adapters are isolated from the core package so that the same compiler and validators run on macOS development machines and Ubuntu RTX nodes.

The current reference implementation generates three IFC4.3 fixtures: connection-plate positioning, pin insertion, and sequential assembly. Each IFC file is accompanied by

machine-readable ground truth and is compiled into an intermediate scene document and a canonical CEWG JSON document. A published JSON Schema constrains its node and edge types, while an independent integrity check rejects duplicate nodes and dangling graph relations. The fixture generator also supports paired deterministic variants of component scale, three-dimensional placement offset, and interface tolerance. Low-discrepancy factor sequences are used so that the benchmark can be regenerated without a hidden random state; component mass changes with the cube of geometric scale to preserve material density.

IFC placements are multiplied by the file’s declared length-unit scale before being written to the meter-normalized scene. This step is tested against generated ground truth because an IFC file expressed internally in millimetres can otherwise produce a syntactically valid USD scene whose object translations are wrong by three orders of magnitude. The USD output consists of a global navigation layer, independently loadable work-zone payloads, task-specific interaction layers, and a root stage that composes them through references and payloads. The frozen v0.2.0 baseline carries thirty versioned rule identifiers covering coordinate normalization, IFC identity, work-zone activation, task references and contracts, robot and tool capabilities, assembly-interface closure, safe payload resolution, and critical-property provenance; v0.3.0 adds two evidence-grounded interface-fidelity rules (CSR-SCN-031/032), 32 in total.

The local benchmark currently implements

30 single-fault operators and five compound faults spanning spatial frames, identity, layer composition, task contracts, embodiment, interfaces, physical validity, construction state, and provenance. Applied to the three fixtures, they produce 105 controlled cases. A strict rule-to-tool registry contains 29 bounded repair operations. One challenge fault, an unsupported model-generated friction value, is intentionally excluded from automatic repair and must be escalated. Every mutation records its rule, tool, changed scene paths, and trusted evidence before the complete validator is rerun. These CPU cases verify the mechanics and measurement code; they do not replace the frozen baseline comparison or the downstream RTX task experiment. The complete implementation, frozen partition manifest, experiment registry, environment lock files, and per-episode ledgers are organized for release; the permanent repository DOI will be inserted at acceptance (see Data and code availability).

6. Experimental design

6.1. Scenarios and tasks

The benchmark is built from parameterized light-steel assembly scenarios: connection-plate positioning, simplified pin insertion, and sequential two-component assembly. The defect-consequence study of this paper is executed at the fixed-base insertion level; whole-body and building-scale mobile-manipulation levels are out of scope for the present results and are stated only as the benchmark’s intended progression (Section 8).

Figure 6 and table 5 specify the intended evidence and were frozen before the experiments were run, which prevents later selection of only visually successful tasks.

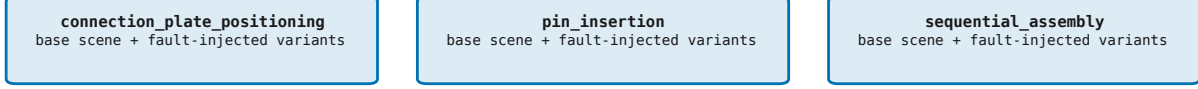
6.2. Fault-injection benchmark

Faults cover semantic identity, geometry and coordinates, physical properties, assembly interfaces, construction states, and task references. Every injected fault has a known location, severity, valid repair, and expected task consequence. The current CPU suite comprises 30 single faults and five compound faults. Development, test, and challenge partitions are frozen before the agent comparison. The challenge partition includes faults that should be rejected or escalated rather than repaired automatically. Consequential findings are recorded separately from the injected root cause; for example, an unresolved robot actor also prevents capability verification.

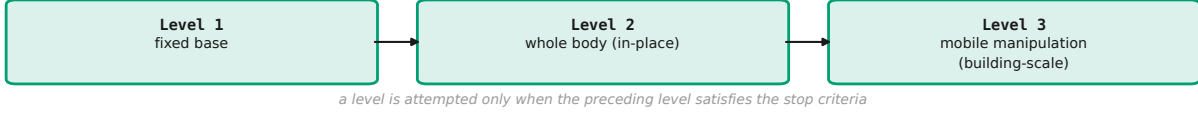
6.3. Baselines and ablations

The baselines are direct IFC geometry conversion, frozen deterministic conversion with fixed repair, and unconstrained agentic repair. The full method combines the scene contract, multi-scale compiler, deterministic validator, provenance, and constrained agent selection from a whitelisted tool registry. All methods receive the same scene and fault, and none may inspect the pristine target scene or fault label during evaluation. Ablations remove provenance, task-level validation, task-activated payloads, typed diagnostics, or agentic tool selection one at a time.

Scenarios (generated IFC fixtures → OpenUSD stages)



Embodiment levels (progression gated by stop criteria)



Fault partitions • comparison methods • freeze

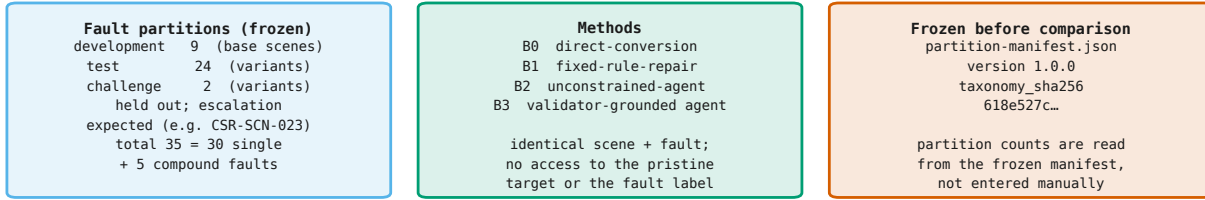


Figure 6: Benchmark organization across construction scenarios, embodiment levels, controlled scene defects, and comparison methods.

6.4. Metrics and statistical tests

Static evaluation measures semantic preservation, relation F1, critical-fault precision/recall/F1, repair success, incorrect repair, human escalation, and runtime. Scene efficiency measures load time, peak memory, active prim count, and collision-query cost. Task evaluation measures reachability, collision-free planning, grasping, alignment, insertion, and complete-task success.

Paired scenes are compared using McNemar tests for binary outcomes and Wilcoxon signed-rank tests for time and iteration counts. Bootstrap confidence intervals quantify uncertainty. The association between the contract readiness signal and downstream task outcome is assessed with a

rank (Spearman) correlation across conditions and a condition-level confusion structure. Language-model methods are repeated with at least five seeds per case, with model version, decoding parameters, token use, and cost retained in the experiment registry. The resampling unit for all confidence intervals is the paired case or episode, not the fault family; defect conditions share paired baseline episodes and are analysed as correlated comparisons. The defect sweep evaluates one primary family-wise hypothesis per family (any effect versus none); secondary severity levels are reported with effect sizes and exact p -values without a family-wise correction claim, and non-significant outcomes are reported as inconclusive rather than as evidence of no

Table 5: Benchmark composition. Faulted-case counts are read from the frozen partition manifest (`data/partition-manifest.json`): 9 development, 24 test, and 2 challenge faults per scenario scene, with test and challenge faults evaluated on held-out geometric variants. All scenarios are exercised end-to-end at the compile–validate–repair level; robot execution in this paper is reported for the pin-insertion scenario at the fixed-base level, and the remaining scenario $\times$ level combinations are the benchmark’s intended progression.

Scenario	Parameterization	Robot task levels	Faulted cases	Primary outcome
Connection-plate positioning	Component size, pose, target height, work-zone obstacle	Fixed base; whole body; mobile	9 dev / 24 test / 2 challenge	Placement pose and state completion
Simplified pin insertion	Interface axis, clearance, friction, initial misalignment	Fixed base (executed here); whole body, mobile (progression)	9 dev / 24 test / 2 challenge	Insertion depth without jam
Sequential assembly	Component order, temporary support, path obstruction, state transition	Fixed base; whole body; mobile	9 dev / 24 test / 2 challenge	Complete legal task sequence

effect (absence of a detectable effect at the
930 available power, not equivalence).

Table 6: Frozen comparison and ablation design. All methods receive identical faulted scenes and are evaluated by the same post-repair validator and runtime tasks.

Method	Construction IR	Multi-scale USD	Deterministic scene rules	Provenance	Repair
Direct conversion	—	—	—	—	—
Fixed-rule conversion	✓	✓	✓	✓	fixed
Unconstrained agent	✓	✓	prose report	partial	agent
Full method	✓	✓	✓	✓	constrained agent

7. Results

7.1. CPU implementation check

The current local run is reported only as a software-verification milestone. Across three source scenes and 35 fault variants per scene, the benchmark created 105 cases containing 141 expected rule findings. The validator ~~re-~~ covered all expected findings without an additional finding under the registered ground truth. The fixed whitelisted-tool implementation returned 102 cases to a valid state and escalated the three unsupported-friction cases. Validation and repair were sub-millisecond in this small CPU check, but those timings vary with host load and are retained in the run report rather than treated as a performance result.

The same implementation check compared the task-activated root with an all-loaded root over three-zone fixtures. Task activation reduced composed prim count by 20–25%, rigid-body and collider count by 40–50%, and bytes in used USD layers by 18.0–24.5%. These are structural consequences of the fixture and confirm that the two composition conditions differ as intended; they are not substitutes for RTX load-time, memory,

and collision-query measurements.

These values should not be interpreted as evidence that the method generalizes to unseen construction scenes. The fault operators were written against the same explicit contract as the validator, and the fixed repair implementation had access to trusted source evidence. Their purpose is to verify rule coverage, compound-fault accounting, audit generation, and the distinction between safe repair and escalation before the held-out and language-model comparisons are executed.

7.2. Compiler fidelity

On the three-scenario fixture suite, the IFC-to-scene compiler preserves the source semantics exactly: component identity recall, IFC GUID accuracy, categorical attribute accuracy, interface-relation and task-binding F1 are all 1.000, with zero position, dimension, and relative-mass deviation (`generated/local-pipeline/reports/semantic-preservation`). These are identity results on fixtures whose ground truth is generated jointly with the IFC source; they verify that the compiler is lossless on the contract’s semantic vocabulary, and they calibrate the fault-injection results that follow—every detection miss

reported below is a property of the method under test, not of information lost in compilation.

7.3. Humanoid task outcomes under scene-model feature defects

The runtime executor for the defect-consequence study is a frozen reinforcement-learning insertion policy on the `Isaac-Assembly-G1-v2` task: a Unitree G1 with a locked lower body transports a virtual-grasped M20 bolt and inserts it along the horizontal axis into the lower-right hole of a 2×2 bolt group on the beam end plate of a two-column portal frame (Figure 7). The plate carries collision geometry with the four holes cut out, so both the hand and the bolt are physically blocked outside the holes; the bolt may pass only through the hole circle. Success follows the same criterion as the training reward: transverse alignment error below 12mm and insertion depth of at least 20mm from the plate surface. The checkpoint used (fine-tuned on this scene) achieves 87.0% success over 200 evaluation episodes without injected defects. Figure 8 shows one successful execution sequence of this executor in the defect-free scene. Table 7 specifies the executor protocol that the defect-consequence conclusions are conditioned on; they are not claims about arbitrary controllers.

Defects from the six feature families of Section 6 were injected at episode reset as controlled deviations between the scene model observed by the policy and the ground-truth insertion target, at graded severities. All conditions share one frozen policy, one seed parti-

tion, and an identical episode-randomization stream: the injection events consume the same amount of randomness whether or not a defect is active, so episode k under any defect condition is the exact paired replicate of episode k under the defect-free baseline. Paired binary outcomes are compared with exact two-sided McNemar tests; b/c denote episodes that changed from success to failure and from failure to success, respectively. Two defect classes are distinguished throughout: *information defects* leave the physical scene intact but corrupt what the controller can know or assume (geometry, semantic identity, task reference, construction state), whereas the *interface-clearance* conditions change the physically executable task itself (an emulation of jamming, not a contact-mechanics simulation). In total 6,200 paired episodes are reported (14 conditions $\times$ 2 seed partitions $\times$ 200 episodes, plus 600 in the repair-closure batch of Section 7.7); every figure and table is generated from the per-episode ledgers by the registered scripts.

Table 8 separates discrete and continuous effect patterns rather than a single criticality ranking. Symbolic and referential features—task references, construction states, and semantic identity—are fatal when *wrong*: a single wrong reference drives success to zero because the executor’s target is defined by them. A *missing* semantic label is the graded exception (12.0% reward, 1.0% geometric-valid), the nominal-pose prior occasionally sufficing, so correctness requirements are binary while completeness requirements can be graded. The clearance conditions form a separate, gated pressure test: shrinking the effec-

Table 7: Executor protocol for the defect-consequence study. All criticality conclusions are conditioned on this observation, training, grasp, and evaluation setup, not on executors in general.

Aspect	Protocol
Observation	State-based frames (<code>ee_frame</code> : hand, bolt, plate/hole relative to torso), arm/hand joint pos/vel, last action; no images or point clouds
Action	24-dim joint-position targets (10 arm + 14 hand)
Grasp	Virtual (kinematic bolt follows palm; no contact force channel)
Defect injection	Episode reset; scene-model observation vs. ground-truth target deviation, graded severities
Target update	Hole coordinate read from the observed frame each step
Success	Reward (12 mm + 20 mm) and geometric-valid (1 mm + 20 mm), evaluated at the same timestep
Termination	Timeout (600 control steps) or bolt-fallen; <i>no</i> success-latch termination, so every episode runs its full protocol and geometric-valid is measured over the complete execution, not truncated by an earlier reward success
Training note	Fine-tuned with ± 5 cm member-pose randomization, which is what makes absolute-pose errors partly correctable

tive radial clearance of the nominal $\text{\O}22$ mm hole from 1 mm to 0.75 mm roughly halves the reward success (87.0% $\rightarrow$ 32.5%), and 0.25 mm is nearly fatal. Because this family acts on the executor through an explicit geometric gate rather than through the information the controller receives, we report it alongside but do not rank it against the information defects (Section 8).

The geometric family separates two effects that are easily conflated, and the answer depends on executor precision. A 10 mm error in the *absolute* pose of the whole connection node is partly compensated by the policy’s closed-loop perception (-4.5 pp, $b/c=10/1$, $p = 0.012$), while the same 10 mm applied to the hole position *relative to the observed member*—the quantity the scene model certifies and the policy cannot see—costs -8.0 pp

($p < 0.001$); 2 and 5 mm hole errors stay inside the reward tolerance band (86.0% and 86.0%, n.s.).

Reward success versus geometric validity. The 12 mm reward threshold is an information-chain criterion, not a certificate of physical insertability: the nominal $\text{\O}22$ mm hole leaves only 1 mm radial clearance for the M20 bolt. Evaluating each episode under a strict *simultaneous* geometric criterion (at one timestep, transverse error within the 1 mm clearance *and* depth at least 20 mm) keeps the strict result a subset of the reward result by construction, and where the two diverge it sharpens rather than softens the contrast above. The defect-free executor reaches only 43.5% geometric-valid against 87.0% reward success—roughly

half of reward successes are motion completions that never achieve the simultaneous geometric bound, a direct measure of how much the 12 mm threshold overstates physical insertability. Anchor-relative hole errors, which the reward criterion partly absorbs, become strongly consequential geometrically: geometric-valid falls to 31.0% at 2 mm, 29.5% at 5 mm and 2.0% at 10 mm, so the certified hole coordinate is the dominant geometric quantity once physical insertability is required. In contrast, absolute member-pose error stays largely correctable (41.5% geometric-valid at 10 mm). A paired McNemar test between the two 10 mm conditions under the geometric criterion confirms the separation is decisive, not marginal ($b=82$ versus $c=3$, $p = 5.3 \times 10^{-21}$). The interface-clearance conditions are unchanged between the two criteria, because their explicit geometric gate already enforces the physical bound. The loose reward threshold therefore masks geometric failure precisely for the information family the scene model is responsible for certifying, which is the empirical reason interface coordinates require certified values with provenance (Section 8).

Physical-property defects (bolt mass $\times 0.5$ and $\times 2$) produced bit-identical outcomes ($b/c=0/0$). This null result is structural, not statistical: the executor’s virtual-grasp scaffold makes the bolt kinematic, so mass and friction have no dynamical channel in this task stage. We therefore report the physical family as *not consequential for the transport-and-insert motion chain* and defer its force-level criticality to a full-contact ablation (Section 8).

All findings replicate under a second seed partition (seed 123, 200 paired episodes per condition): reward success deviates by at most 4.0 pp and geometric-valid success by at most 7.5 pp from the seed-42 values, and effect directions match in every condition. Two borderline cells move across the significance line between partitions—the 10 mm member-pose reward effect is significant on seed 42 ($b/c=10/1$, $p = 0.012$) but not on seed 123 ($b/c=16/8$, $p = 0.15$), and the 2 mm hole geometric effect shows the same fragility ($p = 0.0022$ versus 0.34)—which is why we report effect sizes with confidence statements and treat single-partition borderline cells as inconclusive rather than confirmed (Section 8).

7.4. Detection and repair on the frozen partition

Table 9 reports the registered run over the frozen partition of Section 6 (105 cases: 9 development, 24 test, and 2 challenge faults over three scenario scenes, with test and challenge faults evaluated on held-out geometric variants; manifest `data/partition-manifest.json`, run B-CPU-DETREp-0001). Direct conversion detects none of the scene-level construction faults: its generic structural check cannot name a single rule finding, and it attempts no repair. The frozen rule set with one whitelisted repair action per rule recovers every expected finding on all 105 cases (precision and recall 1.000, no false positives), returns 102 cases to a state equivalent to the pristine scene (97.1%, bootstrap 95% CI [93.3, 100]), and escalates exactly the three challenge cases whose friction record lacks an

authoritative source (2.9%)—the designed behaviour, since fabricating a physical parameter would be an unsupported repair. No repair that passed revalidation deviated from the pristine scene (incorrect repair 0.0%). The detection gap between the two methods is significant on the paired cases (exact McNemar, $b=105$, $c=0$, $p < 10^{-31}$). The language-model rows were run against a frozen self-hosted instruction model (Qwen2.5-Coder-14B-Instruct-AWQ, 5 seeds per case, 525 episodes per method, run B-CPU-LLM-0001). The contrast is stark. Left unconstrained with the same scene and a prose problem report, the agent returns a scene equivalent to the pristine one in only 32.8% of episodes; 22.7% of its repairs pass deterministic revalidation while deviating from the pristine scene—invented identifiers, invented preconditions, and invented physical values that no rule can distinguish from real ones—and it escalates 38.3% of cases, including repairable faults, while 6.3% of replies are unusable outright (mostly truncation at the frozen decoding budget). The validator-grounded agent, restricted to selecting among the same whitelisted tools as the fixed-rule row, reaches 96.8% valid repair versus the deterministic 97.1% ceiling (paired McNemar, $b=2$, $c=0$, $p = 0.5$), with zero incorrect repairs across all 525 episodes. Two qualifications apply. First, $p = 0.5$ is a non-significant difference, not evidence of equivalence: we do not claim non-inferiority, which would require a pre-specified margin. Second, the deterministic fixed-rule row is faster (0.31 ms versus 1.5 s) and at least as accurate, so we position the deterministic contract and its whitelist as the

primary repair method; the language-model agent is an optional, contract-constrained invocation of the same tool registry, whose value is that it can be grounded at all without the unsafe modes of the free-form agent (fixed-rule vs free-form: $p = 3.6 \times 10^{-102}$), not that it outperforms the deterministic core.

Complementing the injected-fault coverage, we also checked the validator against *legitimate but atypical* normal scenes: nine contract-compliant variants of the pristine scene spanning extreme member dimensions, boundary and oversized tolerances, three interface-axis orientations, an extra work zone, an extra reference-closed component, an atypical member placement, and zero-width provenance intervals. All nine pass with zero critical findings (false-rejection rate 0/9), so the contract’s precision is not an artefact of only ever seeing injected faults (`generated/normal-sample-fp/report.json`).

A scale sweep over synthetic floors with 1–20 work zones (four members each) confirms that the composition mechanism, not the fixture size, drives the saving: the task-activated working set grows only with the global layer plus the single active zone (15→91 prims), while the all-loaded composition grows linearly with the zone count (15→205 prims); at 20 zones, task activation touches 56% fewer prims and 55% fewer layer bytes (`resource_efficiency.csv`, run A3-CPU-RES-0001).

7.5. External IFC sanity check

As a check outside the generated fixtures, we compiled the buildingSMART IFC 4.3 ADD2 reference model

`Building-Architecture.ifc` (13 elements, 2 spaces, no task entities; `data/external/`). The compiler produced an empty task scene by design—elements without CSR property-set annotations are skipped rather than guessed—and the validator refused readiness with two critical findings (no mass tolerance evidence records). Unannotated real deliveries are therefore compilable but never silently treated as task-ready; the CSR annotation layer is the explicit bridge, and its absence is itself a contract finding (`generated/external-ifc/report.json`).

To check the complementary direction—that a non-fixture source can become task-ready—we hand-authored an independent beam-plate-pin model (`independent_beam_plate.ifc`), written from scratch rather than by the fixture generator, carrying the CSR annotation layer (one work zone, one interface with mate, tolerance, and axis, one insert task, and per-component mass provenance). It compiled to a non-empty scene (3 components, 1 interface, 1 work zone, 1 task, 6 provenance records) and passed the contract with zero issues (`generated/external-ifc-independent/report.json`).

The two external checks together bound the annotation layer’s role: absent, the scene is empty and refused; present and independently modeled, it compiles and validates end-to-end.

7.6. Readiness scores and the boundary of contract visibility

To test whether the contract’s readiness signal predicts downstream outcomes (hypothesis H4), each humanoid defect condi-

tion of Section 7.3 was re-expressed as its scene-contract equivalent on the matching fixture scenario and scored by the validator (readiness = fraction of the 30 frozen v0.2.0 rules not violated). We assess the confusion structure against the *geometric-valid* outcome of Table 8 (the main assembly metric), not the looser reward criterion, because the contract’s job is to certify information the executor must physically rely on. Every condition the validator flags is catastrophic (semantic identity, task reference, construction state), and in these batches it never flags a condition whose geometric outcome is not significantly degraded. Its blind spot is exactly the geometric family: the 2, 5 and 10 mm hole-coordinate errors and the three reduced-clearance conditions all degrade the geometric-valid outcome yet pass silently. Contract visibility therefore ends exactly at the boundary of *declared* tolerances: the validator checks that tolerances and references exist and are well-formed, but not that the coordinates inside them are faithful.

The mapping is by failure predicate, not by scene identity: each runtime defect is re-expressed as the contract fault that shares its predicate—a commanded or referenced value disagrees with the task-intended one—and the sensitivity figures below are therefore a condition-level *visibility analysis on contract-equivalent faults*, not a claim that the validator inspects simulator states. One mapping deserves explicit statement. The runtime task-reference defect repoints the executor’s aim to a sibling hole of the 2×2 group: in the rig each hole is an individually addressable interface target, so the commanded tar-

get is a valid scene location that is not the task-intended one. The fixture expresses the same wrong-target predicate at component granularity: the injection keeps the task’s interface reference (`plate_pin_insertion`, owned by `connection_plate`) but repoints `target_component` to the mate component (`assembly_pin`), and CSR-SCN-028 fires precisely because the task target no longer owns the referenced interface. The concrete record—target component, interface identifier, owner, and the rule’s diagnostic path—is the Defect A column of Table 15.

The condition-level confusion structure is derived from the per-condition geometric statistics in Table 13, which report both seed partitions in full: geometric-valid successes in the defect-free versus defect scene, the discordant pairs, and the exact McNemar p for each of the 13 defect conditions. A condition is counted *consequential* when that test is significant ($p < 0.05$); conditions without a significant paired drop are counted as *no significant degradation detected in this batch*, which is not proof of no effect. The resulting confusion counts, summarised in Table 14, are TP=4, FN=6, TN=3, FP=0 on seed 42 (sensitivity 4/10 at the *condition* level—not a per-episode classification rate, which would be misleading given the few independent conditions) and TP=4, FN=5, TN=4, FP=0 on seed 123 (sensitivity 4/9): the borderline 2 mm hole shift is significant on seed 42 ($p = 0.0022$) but not on seed 123 ($p = 0.34$), so that single cell moves between FN and TN across partitions. The validator raises no false alarm in either batch (3/3 and 4/4); we report these as observed batch counts rather than a general

zero-false-alarm guarantee, noting that two of the negative-class conditions are mass perturbations whose dynamics channel is closed by the kinematic virtual grasp. Note the contrast with the reward criterion, under which the 2 and 5 mm hole errors looked absorbed (TN): the geometric metric reveals them as consequential, which is exactly the blind spot the new rules below are meant to close. Two kinds of “failure” must be kept apart: an FN here is a *task-risk* miss (the scene runs degraded but the contract does not see it), distinct from a *source-consistency* violation (the scene is unfaithful to its evidence yet still within the executor’s tolerance), a distinction that matters when the new rules are scored (Section 8). As an exploratory association, the Spearman rank correlation between readiness and condition-level success is $\rho = 0.58$ across the 13 conditions; readiness takes only two distinct values here (four flagged conditions at 0.967, nine silent at 1.0), so the coefficient reduces to a two-group contrast, to which we attach no p -value—the confusion structure above is the primary analysis.

Table 13: Per-condition geometric-valid statistics behind Table 14: 200 paired episodes per condition and seed partition, defect-free versus defect scene geometric-valid successes (ok→def), discordant pairs b/c (lost/gained), and exact two-sided McNemar p ; (a) seed 42, (b) seed 123. *Validator* names the rule that fires on the condition’s contract-equivalent fault (v0.2.0; mapping in the text); *silent* conditions violate no declared rule. Cells: TP = flagged and consequential; FN = silent but consequential; TN = silent, no significant degradation detected in the batch; FP = flagged without detected degradation (none occurred). Generated from the per-episode ledgers of Table 8.

(a) seed 42						
Condition	rule	ok→def	b/c	p	cell	
hole +2 mm	silent	87→62	44/19	0.0022	FN	
hole +5 mm	silent	87→59	53/25	0.0020	FN	
hole +10 mm	silent	87→4	85/2	4.9e-23	FN ¹³⁸⁵	
member +10 mm	silent	87→83	29/25	0.68	TN	
clear. 0.75 mm	silent	87→65	41/19	0.0062	FN	
clear. 0.5 mm	silent	87→41	52/6	3.2e-10	FN	
clear. 0.25 mm	silent	87→15	74/2	7.7e-20	FN	
sem. missing	SCN-003	87→2	85/0	5.2e-26	TP	
sem. wrong	SCN-004	87→0	87/0	1.3e-26	TP	
task ref.	SCN-028	87→0	87/0	1.3e-26	TP ¹³⁹⁰	
state	SCN-021	87→0	87/0	1.3e-26	TP	
mass ×0.5	silent	87→87	0/0	1.0	TN	
mass ×2	silent	87→87	0/0	1.0	TN	

(b) seed 123						
Condition	rule	ok→def	b/c	p	cell	
hole +2 mm	silent	86→77	39/30	0.34	TN ¹³⁹⁵	
hole +5 mm	silent	86→62	53/29	0.011	FN	
hole +10 mm	silent	86→11	82/7	2.4e-17	FN	
member +10 mm	silent	86→74	41/29	0.19	TN	
clear. 0.75 mm	silent	86→57	42/13	1.1e-4	FN	
clear. 0.5 mm	silent	86→36	58/8	1.8e-10	FN	
clear. 0.25 mm	silent	86→11	76/1	1.0e-21	FN	
sem. missing	SCN-003	86→0	86/0	2.6e-26	TP ¹⁴⁰⁰	
sem. wrong	SCN-004	86→0	86/0	2.6e-26	TP	
task ref.	SCN-028	86→0	86/0	2.6e-26	TP	
state	SCN-021	86→0	86/0	2.6e-26	TP	
mass ×0.5	silent	86→86	0/0	1.0	TN	
mass ×2	silent	86→86	0/0	1.0	TN	

Table 14: Condition-level confusion summary of the contract against the paired geometric-valid outcome, derived from Table 13 (13 defect conditions, baseline excluded). *Consequential* = significant paired geometric degradation (exact McNemar $p < 0.05$); the negative class is “no significant degradation detected in this batch”, not proof of no effect.

Seed	TP	FN	TN	FP	Sensitivity	Specificity
42	4	6	3	0	4/10	3/3
123	4	5	4	0	4/9	4/4

We converted this blind spot into two contract rules and verified them on an independent test set. CSR-SCN-031 requires member-relative interface coordinates to carry an evidence record and to match it within the declared tolerance; CSR-SCN-032 requires interface clearance to exceed the fastener envelope by the task-required minimum. On held-out scene variants never used during rule development (variant index 2), contract v0.3.0 passes pristine scenes and within-tolerance (1 mm) coordinate drift, and detects both 5 mm evidence-breaking drift and sub-minimum clearance on every scene, while the frozen v0.2.0 contract is confirmed blind to the same defects (`generated/contract-v030/report.json`). The complete loop—contract, runtime measurement of its blind spot, contract extension, independent verification—is itself part of the method: the humanoid benchmark serves as a calibration instrument for scene-readiness contracts.

1405

7.7. End-to-end traceability of representative defects

To make the chain from IFC to task outcome auditable rather than asserted, Table 15 traces two representative defects through every pipeline stage on the `pin_insertion` fixture, using committed artefact identifiers. Defect A (`mismatched_task_interface_target`) is caught at validation: the interface reference closes, but the task binding points at the wrong interface, so the deterministic task-reference rule flags it and the executor is fatally mis-targeted. Defect B (a 5 mm member-relative hole-coordinate drift) passes the v0.2.0 contract silently, yet the executor still degrades—this is the blind spot the added rule CSR-SCN-031 now closes. In both cases the same IFC GUID, CEWG node, composed prim, resolved value, validation record, runtime target, and outcome are recoverable from the ledger, so each number in the defect benchmark can be traced back to a specific scene artefact rather than to a defect-class label.

Closing the loop on Defect B, the correct-fault-repaired comparison of Section 7.7 was executed as one protocol batch (R-REPAIR-GEO-0001: same frozen v2c checkpoint `model_16746.pt`, same seed 42, 200 paired episodes per condition, same independent reward and geometric evaluators; per-episode records in `data/results/repair_closure*_s42.jsonl`). Repair is applied *before* execution, not during it. In every episode of the faulted group, the episode-reset event writes the drifted coordinate into the scene model (hole refer-

ence +5 mm); the repaired group receives the same injection, which is then repaired at episode start, before the policy’s first action: the coordinate is re-derived from the evidence record, written back to the simulator, and settled with two zero-action control steps. No state is reset after repair, no repair occurs mid-execution, and the controller, checkpoint, seed, and evaluation window are identical across the three groups. The faulted scene degrades the task (reward 84.0%, geometric-valid 22.5%): the paired geometric drop is strongly significant ($b/c=57/15$, $p = 6.5 \times 10^{-7}$) while the paired reward drop is not ($b/c=7/1$, $p = 0.070$)—the masking pattern of Table 14 reproduced inside a single repair chain. Re-deriving the coordinate from the evidence record recovers the task (reward 88.5%, geometric-valid 34.0%), and the fault-to-repaired gain is significant on both criteria ($b/c=1/10$, $p = 0.012$ reward; $b/c=20/43$, $p = 0.005$ geometric). Against the defect-free scene, no significant difference in reward success is detected ($b/c=3/6$, $p = 0.51$), but a significant geometric residual remains ($b/c=38/19$, $p = 0.016$). At execution start the repaired group differs from the defect-free group only in the history of the reference shift and the two settling steps; whether the residual stems from that settling transient or from run-to-run variation is not separately identified by this experiment, so we report it as an unresolved residual rather than attributing it to a mechanism. The same limitation bounds attribution of the gain: the repair intervention is the coordinate restoration *and* the two settling steps together, so the

experiment supports the effect of the complete pre-execution repair protocol, not of the coordinate edit alone. We therefore state the narrower, sufficient claim: under this protocol—evidence-driven coordinate restoration plus settling—geometric-valid success rises from 22.5% to 34.0%, a significant partial recovery of 11.5 of the 21.0 pp geometric loss ($\approx 55\%$), while the compile-validate-repair chain is shown to remove the information defect itself, since the runtime target of the repaired scene is the true hole again.

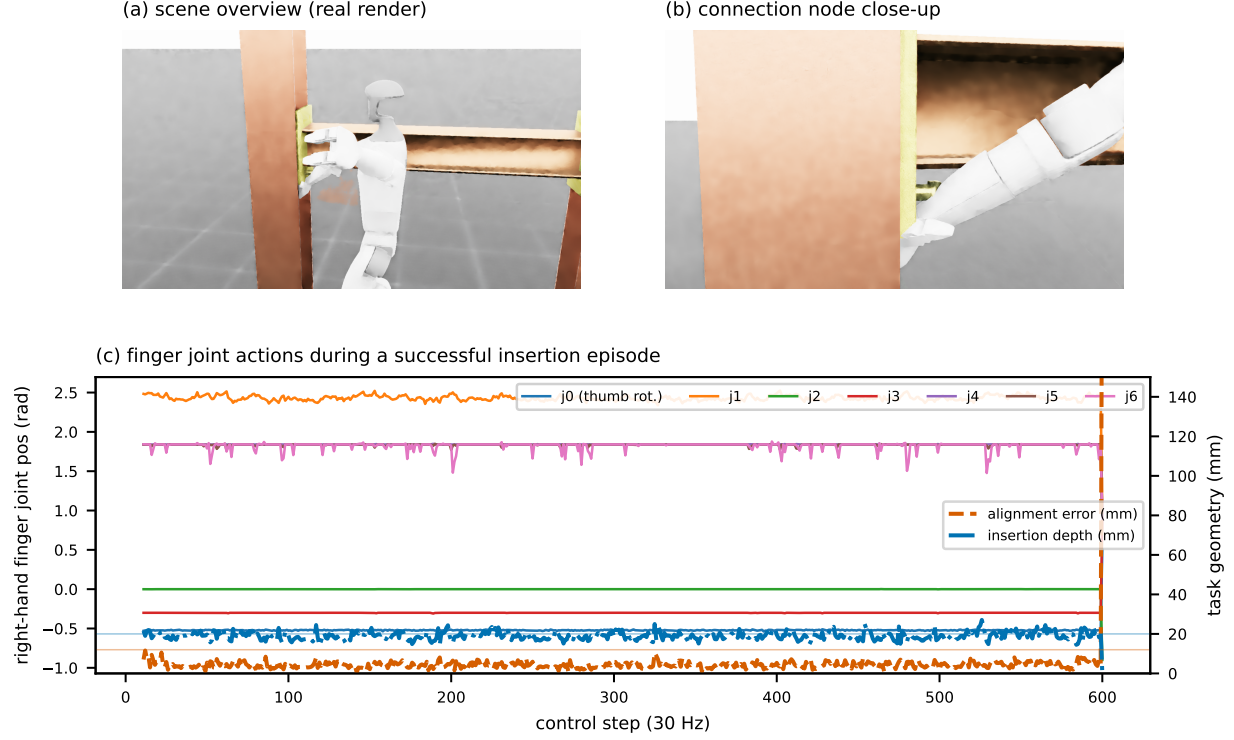


Figure 7: Experimental setup and executor action stream. (a)–(b) True simulator renders of the two-column portal-frame assembly scene: the overview shows the G1 working at the end plate of the supported beam; the close-up shows the 2×2 bolt group on the collision-enabled plate. (c) Right-hand finger joint positions and task geometry over one successful insertion episode: the policy closes the fingers early and holds a power grasp while the transport–align–insert phases proceed; the alignment error (dashed) and insertion depth (dash-dot) reference lines use the right axis. The sharp terminal jump at the right edge is the environment reset at the timeout boundary (600 control steps): under the study protocol episodes are not terminated early on success, so the grasp is held through the full window and the jump is the reset repositioning, not a control artefact. Frames and trajectories are generated by the registered capture scripts.

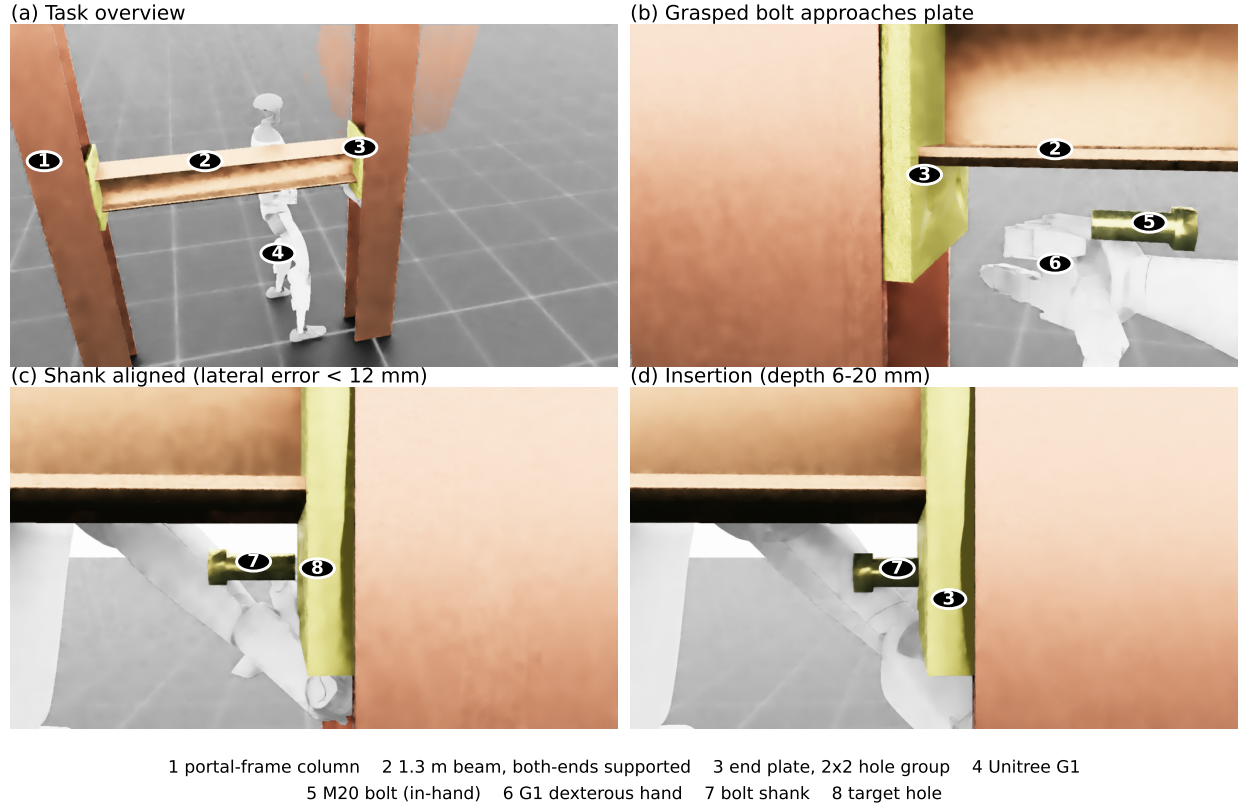


Figure 8: Executed insertion sequence of the frozen policy in the defect-free scene (true simulator renders, captured at 2560×1440). (a) Task overview: the G1 stands at the end plate of the 1.3m beam, which is supported at both ends by the two-column portal frame. (b) The right hand transports the virtually grasped M20 bolt toward the plate. (c) The shank is aligned with the lower-right hole of the 2×2 group (transverse error below 12 mm). (d) Insertion proceeds along the hole axis; success requires a depth of at least 20 mm from the plate surface. This example episode satisfies the 12 mm reward criterion; the separate 1 mm geometric-valid subset of such episodes is analysed in Table 8. Numbered markers identify the scene features listed in the legend at the bottom of the figure. All frames are captured from one successful evaluation episode by the registered panel-capture script.

Table 8: Insertion outcome under injected scene-model feature defects (200 paired episodes per condition, seed 42). Both criteria are evaluated *at the same timestep* of each episode and aggregated identically, so the stricter criterion is always a subset of the looser one. *Reward success*: at some timestep transverse error below 12 mm and depth at least 20 mm (the training-reward criterion). *Geometric valid*: at the *same* timestep transverse error within the 1 mm nominal radial clearance *and* depth at least 20 mm—the strict sub-criterion; with the insertion axis and bolt held fixed in this rig, alignment and depth are the operative geometric degrees of freedom. Δpp is the reward-success change relative to the defect-free baseline of the same batch. Significance: exact McNemar test on reward success, $***p < 0.001$, $**p < 0.01$, $*p < 0.05$.

Feature family	Injected defect	Reward success	Geometric valid	Δpp	b/c	p
—	none (baseline)	87.0%	43.5%	—	—	—
Geometry, hole position	± 2 mm, random direction	86.0%	31.0%	−1.0	4/2	n.s.
Geometry, hole position	± 5 mm	86.0%	29.5%	−1.0	3/1	n.s.
Geometry, hole position	± 10 mm	79.0%	2.0%	−8.0	19/3	***
Geometry, member pose	whole node +10 mm	82.5%	41.5%	−4.5	10/1	*
Assembly face	radial clearance 0.75 mm	32.5%	32.5%	−54.5	110/1	***
Assembly face	radial clearance 0.5 mm	20.5%	20.5%	−66.5	133/0	***
Assembly face	radial clearance 0.25 mm	7.5%	7.5%	−79.5	160/1	***
Semantic identity	label missing (nominal prior only)	12.0%	1.0%	−75.0	150/0	***
Semantic identity	label wrong (plate centre, no hole)	0.0%	0.0%	−87.0	174/0	***
Task reference	wrong hole of the 2×2 group	0.0%	0.0%	−87.0	174/0	***
Construction state	member not yet in place	0.0%	0.0%	−87.0	174/0	***
Physical properties	bolt mass $\times 0.5$ / $\times 2$	87.0%	43.5%	+0.0	0/0	n.s.

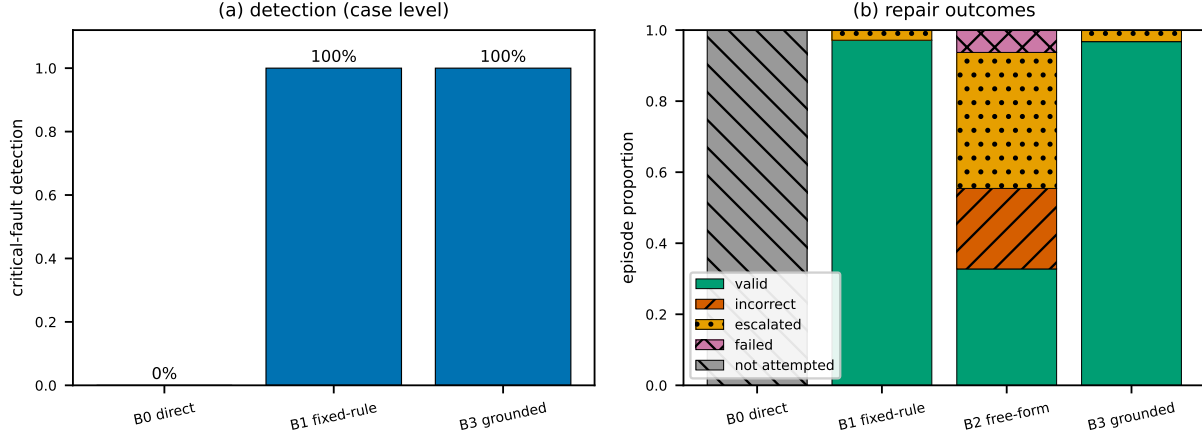


Figure 9: Detection and repair performance across the frozen partition (105 cases; agent methods 5 seeds per case). (a) Case-level critical-fault detection by method; the free-form agent produces no typed findings (n/a, see text). (b) Repair-outcome composition per method. The validator-grounded agent matches the deterministic ceiling with zero incorrect repairs; the unconstrained agent’s incorrect and over-escalation shares are the unsafe modes the contract eliminates. Colour-blind-safe palette with redundant hatching; generated from the registered result files by `scripts/make_figures.py`.

Table 9: Primary detection and repair outcomes over the frozen partition (105 cases; agent methods: 5 seeds per case, 525 episodes; bootstrap 95% CIs in brackets). Rule-level precision and recall are computed over 141 expected findings per method. Direct conversion has no scene-rule detector, so its detection cells are not applicable. Time is the mean validation-plus-repair time per case (agent rows: mean episode wall time including model inference). ^b The free-form agent produces no typed findings; its observable is repair, scored by independent deterministic revalidation. ^c The full method receives typed validator findings as input; detection equals the fixed-rule row by construction.

Method	Precision	Recall	Critical recall	Valid repair	Incorrect repair	Escalation	Time
Direct conversion	n/a	n/a	n/a	n/a	n/a	n/a	<0.01 ms
Fixed-rule conversion	1.000	1.000	1.000	97.1% [93.3, 100]	0.0%	2.9%	0.31 ms
Free-form agent	n/a ^b	n/a ^b	n/a ^b	32.8% [28.8, 36.8]	22.7%	38.3%	20.0 s
Full method	1.000 ^c	1.000 ^c	1.000 ^c	96.8% [95.2, 98.1]	0.0%	3.2%	1.5 s

Table 10: Repair outcome by frozen partition (development 27 / test 72 / challenge 6 cases; agent rows 5 seeds per case). Development cases are repaired to pristine equivalence by both grounded methods and test cases nearly all (fixed-rule 100.0%, constrained agent 99.4%); the challenge cases—whose repair would exceed the available engineering evidence—are split three repaired and three escalated by design, and never incorrectly repaired. The unconstrained agent fails across all partitions with a substantial incorrect-repair share.

Method	Dev valid (27)	Test valid (72)	Challenge valid (6)	Challenge escalated
Fixed-rule conversion	100.0%	100.0%	50.0%	50.0%
Validator-grounded agent	100.0%	99.4%	50.0%	50.0%
Unconstrained agent	40.0%	32.8%	0.0%	80.0%

Incorrect repair is 0.0% in every partition for the fixed-rule and validator-grounded rows; the unconstrained agent’s incorrect share is 19.3%/24.2%/20.0% (dev/test/challenge).

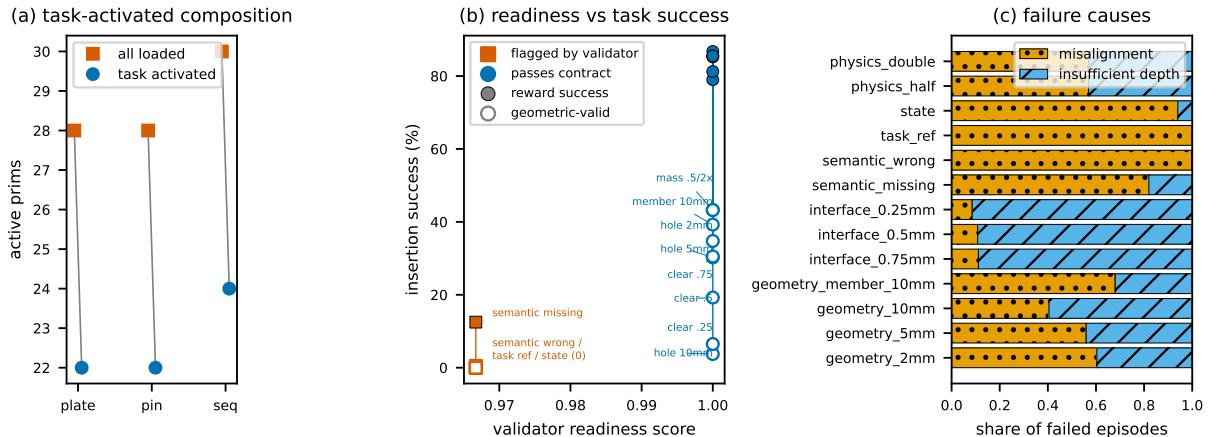


Figure 10: Resource effect of task-activated composition and the relationship between scene readiness and downstream humanoid assembly outcomes. (a) Active prim count per held-out scene, all-loaded versus task-activated composition. (b) Validator readiness score versus insertion outcome for the 13 defect conditions (mean of two seed partitions). Filled markers are reward success; open markers are the simultaneous geometric-valid outcome, joined to the reward value of the same condition by a thin segment. The contract flags every catastrophic condition (squares) but passes the geometric and interface conditions whose geometric-valid outcome collapses while reward success stays high—the gap between the two marker sets is the blind spot quantified in Table 14. (c) Failure-cause composition per defect condition from the per-episode ledger. Generated from `resource_efficiency.csv`, `task_outcomes.csv`, and the readiness-link report by `scripts/make_figures.py`.

Table 11: Runtime resource use and task-level outcomes. Resource cells are reported per composition mode on the held-out variant scenes (structural metrics are exact; load time is the median of five cold opens and varies with host). Peak memory stayed below the 1 MB measurement threshold. Task stages are measured with the frozen RL executor on the defect-free scene (two seed partitions); planning, grasp, and alignment are not instrumented separately under the virtual-grasp scaffold (Section 7.3).

Condition	Active prims	Peak memory	Load time	Planning	Grasp	Alignment	Insertion	Full task
Direct/all loaded	28–30	<1 MB	1.4 ms	n/a	n/a	n/a	n/a	n/a
Fixed rule	identical compiler output as full method				n/a	n/a	n/a	n/a
Free-form agent	method-independent compilation output; task cells not instrumented (Section 7.3)				n/a	n/a	n/a	n/a
Full/task activated	22–24	<1 MB	1.5 ms	n/a	n/a	n/a	87.0% (seed 42)	= insertion

Table 12: Ablation and sensitivity analysis relative to the fixed-rule row of Table 9 (105 paired cases each; bootstrap 95% CIs in brackets). The first four columns are measured on the ablated pipeline itself. The last column is *not* an ablation effect: it is the index of the *related* defect-consequence condition from Section 7.3 that conceptually matches the removed information, shown only to indicate how consequential that information family is at runtime (removing provenance is not the same as corrupting a physical value, and removing the task validator is not the same as every scene having a wrong task reference).

Ablation	Removed information or control	Critical recall	Valid repair	Resource use	Related defect outcome
No provenance	Source authority and uncertainty	0.894 [0.839, 0.941]	85.7% [79.0, 92.4]	unchanged	87.0% (physics, non-consequential)
No task validator	Task binding and action preconditions	0.787 [0.706, 0.868]	80.0% [72.4, 87.6]	unchanged	0.0% (task reference, fatal)
All layers loaded	Task-triggered payload activation	1.000	97.1%	+27% prims, +20% layer bytes	n/a
Fixed repair order	Agentic tool selection	1.000	97.1%	unchanged	identical to fixed rule
Free-form diagnostics	Typed diagnostic schema	1.000 (given)	89.0% [86.3, 91.6]	unchanged	n/a

Table 15: End-to-end trace of two representative defects on the `pin_insertion` fixture. Every stage is recoverable from committed artefacts (IFC, CEWG, scene JSON, validation report, episode ledger). Defect B outcome rows are the paired `R-REPAIR-GEO-0001` batch; fault-to-repaired gains are significant on both criteria ($p = 0.012$ reward, $p = 0.005$ geometric), with a significant residual against the defect-free scene on the strict criterion ($p = 0.016$) discussed in the text.

Pipeline stage	Defect A: wrong interface target (detected)	Defect B: 5 mm hole-coordinate drift (initially silent)
IFC source	IFCPLATE 2LiDpsq0XSP0GZ\$amUn_RZ + interface pset	same plate + IFCMECHANICALFASTENER 1NsEaARyrT8wK7EqXqgqhz
CEWG node	interface:plate_pin_insertion + task:insert_pin	interface:plate_pin_insertion (origin [2.1,1.5,1.05], axis [1,0,0])
Composed prim	/World/zone_a/connection_plate	/World/zone_a/connection_plate
Resolved value	target_component: connection_plate → assembly_pin (interface reference plate_pin_insertion unchanged)	hole coordinate shifted 5 mm off evidence record
Validation	task-reference rule flags (critical)	v0.2.0 silent; caught by CSR-SCN-031 in v0.3.0
Repair	escalate (unsupported retarget)	re-derive coordinate from evidence record
Runtime target	policy aimed at wrong hole	policy aimed 5 mm off true hole
Task outcome (correct scene)	—	reward 87.0%, geometric-valid 43.5% (<code>R-REPAIR-GEO-0001</code> baseline)
Task outcome (fault scene)	0.0% success (fatal)	reward 84.0%, geometric-valid 22.5%
Task outcome (repaired scene)	—	reward 88.5%, geometric-valid 34.0% (repaired before execution; significant gain, significant residual—see text)

(a) faulted evidence record (friction)

```
{
  "entity": "default_steel_contact",
  "property": "friction",
  "source": {
    "benchmark_protocol": "steel_contact_v0.1",
    "source_type": "unverified_model_generation",
    "value": 0.5,
    "unit": "dimensionless",
    "confidence": 0.2,
    "uncertainty_interval": [
      0.3,
      0.7
    ]
  },
  "authority_level": 1,
  "auto_repair_allowed": false,
  "requires_human_confirmation_for_deployment": true
}
```

(b) typed validator finding (CSR-SCN-023)

```
{
  "rule_id": "CSR-SCN-023",
  "severity": "critical",
  "path": "$.provenance[7].source_type",
  "message": "A critical physical property cannot rely solely on an unverified generated value."
}
```

(c) validator-grounded agent escalation

```
{
  "method": "validator-grounded-agent",
  "case": "connection_plate_positioning__v001_unsupported_generated_friction",
  "outcome": "escalated",
  "episodes_escalated": "15/15 (3 scenes x 5 seeds)",
  "note": "no authoritative friction source; repair refused rather than invented"
}
```

Figure 11: Auditable failure–diagnosis–escalation sequence for the held-out challenge case `unsupported_generated_friction (connection_plate_positioning__v001)`. (a) The faulted provenance record: friction entered by unverified model generation with low confidence and repair blocked by policy. (b) The deterministic typed finding (CSR-SCN-023). (c) The frozen validator-grounded model escalates in all 15 episodes rather than inventing a friction value. All panels render committed artefacts (scene JSON, validation report, episode ledger) via `scripts/make_case_study_figure.py`.

8. Discussion

The discussion separates three claims: whether the contract detects construction-specific scene defects, whether constrained repair resolves those defects reliably, and whether improved readiness transfers to run-time task outcomes. This separation prevents successful simulation videos from being treated as evidence that every scene requirement is correct. The defect-consequence study of Section 7.3 speaks to the third claim and, by inversion, to which scene-model features the first two claims must protect.

8.1. Which scene-model features are task-critical

The paired defect-injection results yield a differentiated consequence structure across information families—discrete-fatal reference defects, tolerated member-pose errors, masked hole-coordinate errors, and explicitly gated clearance effects—that is not aligned with the effort distribution of current simulation-readiness specifications, which emphasize asset-level geometry and physical fidelity.

Referential integrity is discrete-fatal. A single wrong task reference, a stale construction state, or a wrong semantic label drove task success from 87.0% to exactly zero under both seed partitions. These features define *what* the executor is asked to do; no amount of perceptual or control robustness can compensate for executing the wrong target. Scene contracts should therefore treat

reference closure—every task reference resolves to an existing, correctly labelled, in-place member—as a hard acceptance gate rather than a soft quality score. Notably, a *missing* semantic label is far less harmful than a *wrong* one (12.0% vs. 0.0%): the executor’s nominal-pose prior occasionally suffices, so completeness requirements can be graded while correctness requirements cannot.

Assembly-interface tolerance is a sensitive continuous feature, gated explicitly. Reducing the effective radial clearance of a nominal Ø22 mm hole from 1 mm to 0.75 mm—an error well within what manual modelling or careless unit handling can introduce—collapsed success from 87.0% to 32.5%, and a 0.25 mm clearance is nearly fatal (7.5%). This is a geometric-feasibility *gate pressure test*, distinct from the information defects: it changes the physically executable task through an explicit geometric gate rather than corrupting what the controller knows. Two qualifications apply. First, the reduced clearance acts on the executor through an explicit geometric gate (the per-episode record shows insertion travel truncated from ≈50 mm to below 5 mm); this emulates the kinematic consequence of jamming but is not a contact-mechanics simulation, so the threshold values should not be read as physical measurements. Second, the 12 mm alignment tolerance of the reward criterion is much looser than the 1 mm nominal clearance: the criterion certifies the information chain, while physical insertability would require the stricter bound. Interface dimensions therefore require certified values with

provenance; an interface field without provenance should be treated as absent.

1605

Anchor-relative accuracy is masked by the reward criterion, not absent.. A 10 mm error in the hole position *relative to the observed member*—the quantity the scene model certifies and the policy cannot see—looked at most tolerable under the reward criterion (−8.0 pp), and 2 or 5 mm looked absorbed (−1.0 pp, n.s.). Under the simultaneous geometric-valid criterion the same family is exposed as the dominant one: valid insertion falls to 31.0% at 2 mm, 29.5% at 5 mm and 2.0% at 10 mm. By contrast, a 10 mm *absolute* shift of the whole connection node stays largely correctable (41.5% geometric-valid, −2.0 pp, n.s.), and a paired McNemar test between the two 10 mm conditions is decisive ($p = 5.3 \times 10^{-21}$). The 12 mm reward threshold thus does not merely loosen the metric—it specifically hides errors in the one geometric quantity the scene model is responsible for certifying. Coordinate guarantees should accordingly be specified and verified *relative to the features the controller can observe*, since absolute-frame accuracy is partially self-correcting at runtime while anchor-relative error is invisible to the controller by construction—and the safe band narrows as executors get more precise.

Physical-property criticality is stage-dependent.. Bolt mass perturbations of $\times 0.5$ and $\times 2$ produced bit-identical outcomes because the transport-and-insert executor holds the bolt in a kinematic virtual grasp. We read this as a structural property of the

motion stage, not evidence that physical fidelity is unimportant: the same defect is expected to become consequential in any stage with forceful contact, and a full-contact ablation of the physical family remains required before it can be ranked.

8.2. Sensitivity and robustness of the findings

Three analyses bound the statistical fragility of these conclusions.

First, all fourteen conditions were re-run under an independent seed partition (seed 123, 200 paired episodes per condition). Reward success deviates by at most 4.0 pp and geometric-valid success by at most 7.5 pp from the seed-42 values, and effect directions match in every condition. Every strong effect replicates decisively—the 10 mm hole error and all three clearance conditions at $p < 10^{-3}$, the catastrophic semantic and referential defects at $p < 10^{-25}$ —and the member-pose and physics conditions remain non-significant. Only the borderline 2 mm hole shift loses its seed-42 significance ($p = 0.34$ against 0.002), which is consistent with a small effect near the detection limit of 200 paired episodes; we treat that cell as inconclusive rather than confirmed. The qualitative criticality structure is otherwise a property of the scene-model features, not of a particular random draw of episodes.

Second, the severity gradients are near-monotone where the defect acts through a continuous channel. Reward success gives 86.0%, 86.0%, and 79.0% for 2, 5, and 10 mm hole-position error against an 87.0% baseline, and 32.5%, 20.5%, and 7.5% for 0.75, 0.5, and 0.25 mm interface clearance; the

geometric-valid gradient is strongly monotone (31.0%, 29.5%, 2.0% for the same hole severities). Near-monotone dose-response supports a causal interpretation of the injected defect rather than threshold noise in the success criterion.

Third, the geometric-valid criterion sharpens the borderline band that the reward criterion leaves ambiguous. The 10 mm member-pose shift is already significant on reward success at $n=200$ ($p = 0.012$), while the 2 and 5 mm hole shifts only become significant once geometric-valid is used ($p = 0.002$ and $p = 0.002$). We accordingly report effect sizes alongside significance and do not rank features on p -values alone; non-significant results are reported as inconclusive at the available power, not as evidence of no effect.

8.3. Boundary of pure-simulation conclusions

All runtime evidence in this study is simulated, and the conclusions should be read as structural rather than numerical. Rigid-body simplification, approximate contact parameters, and simplified pins in place of threaded fasteners mean that absolute success rates and clearance thresholds will shift on hardware. What this study establishes is the *channel* by which information defects enter task outcomes under the specified executor and evaluation protocol; those channels can guide verification experiments for other executors, but their magnitudes, ordering, and tolerance bands require re-calibration rather than direct transfer. Three further boundaries apply. (i) A single robot-controller combination was used: the compensation

of whole-node pose error depends on the executor’s ± 5 cm pose domain randomization during training, and an executor without such randomization would likely show a larger member-pose effect. (ii) The physical-family null result is an artefact of the virtual-grasp scaffold and bounds only the transport-and-insert stage. (iii) The success criterion (12 mm alignment, 20 mm insertion depth) matches the training reward; stricter assembly tolerances would shift all conditions downward but preserve pairing. A physical-robot validation study is deferred to future work; within these boundaries, a defect that is benign in this study is *not* certified benign on hardware, while a defect that is fatal here is fatal by construction of the task definition. No lower-bound relation between simulated and hardware consequences is claimed.

8.4. Implications for cross-runtime specifications

The defect-consequence profile suggests which validation rules belong in a runtime-agnostic scene contract and which require per-runtime calibration. Reference closure, semantic-identity correctness, and construction-state consistency were fatal under the tested executor, and are expected to be so for any controller that consumes *certified* references rather than re-deriving targets by perception; these can be specified once, as cross-runtime rules with hard-fail semantics. Interface-tolerance and anchor-relative-geometric rules have executor-dependent thresholds (here ≈ 1 mm clearance and ≈ 5 mm anchor-relative position) and should be specified parametrically,

with the threshold supplied by the target controller’s tolerance band and verified by the kind of paired injection benchmark used here. Asset-level physical fidelity, finally, should be required only for task stages with a dynamical channel; mandating it uniformly imposes authoring cost without measurable task benefit.

9. Conclusion

This paper frames robot-ready construction simulation as a scene compilation and validation problem rather than a file-conversion problem. It proposes a multi-scale IFC-to-OpenUSD pipeline, a construction task-scene contract, and a deterministically grounded repair process. On a frozen fault-injection partition the contract validator recovered every expected finding without false positives and repaired 97.1% of cases to pristine-scene equivalence, escalating exactly those cases whose repair would exceed the available engineering evidence. Grounding a language-model agent in the contract preserved this reliability (96.8% valid, zero incorrect repairs), whereas the same model without the contract produced validator-passing but incorrect scenes in nearly a quarter of episodes—direct evidence that scene-level engineering authority, not model capability, is the binding constraint on agentic scene repair.

The humanoid defect benchmark closes the loop between scene models and task outcomes. Measured over 6,200 paired insertion episodes, *wrong* values of symbolic and differential features—semantic identity, task ref-

erences, construction states—were discretely fatal, while a *missing* semantic label still degraded outcomes; and the 12 mm reward criterion was shown to mask anchor-relative interface errors: a 10 mm hole-coordinate error left reward success at 79.0% while geometric-valid insertion collapsed to 2.0%, whereas the same 10 mm absolute member shift stayed largely correctable (geometric-valid 41.5%). Cross-checking these outcomes against the validator showed no false alarms but condition-level sensitivities of only 4/10 and 4/9 across the two seed partitions: the contract sees every catastrophic defect family yet remains blind to sub-tolerance geometric and interface fidelity, which motivates the two candidate rules registered in this work and, more broadly, positions runtime task measurement as a calibration instrument for scene-readiness contracts.

The results are bounded by a single executor, a single assembly task, a kinematic grasp scaffold, and simulation-only evidence; extending the defect benchmark to contact-rich stages and additional embodiments, and completing the whole-body task levels, is the natural next step. The framework’s artefacts—the frozen partition, registered scripts, and per-episode ledgers—are intended to make every number in this paper regenerable.

CRediT authorship contribution statement

Zhu Aiyu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—

original draft, Writing—review & editing, Visualization, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data and code availability

The benchmark partition manifests, evaluation scripts, per-episode ledgers, and figure-generation code are available at <https://github.com/aizhuayu/construction-scene-ready>; the robot simulation environments, training configurations, and capture scripts are available at <https://github.com/aizhuayu/g1-steel-assembly>. No proprietary data were used.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used generative AI coding assistants for simulation scripting, experiment orchestration, and language editing. The author reviewed and edited all outputs and takes full responsibility for the methods, evidence, and conclusions of this publication.

References

- [1] S. Kim, M. Peavy, P.-C. Huang, K. Kim, Development of BIM-integrated construction robot task planning and simulation system, *Automation in Construction* 127 (2021) 103720. doi:10.1016/j.autcon.2021.103720.
- [2] A. Zhu, P. Pauwels, B. De Vries, Component-based robot prefabricated construction simulation using IFC-based building information models, *Automation in Construction* 152 (2023) 104899. doi:10.1016/j.autcon.2023.104899.
- [3] H. Oyediran, W. Turner, K. Kim, M. Barrows, Integration of 4d BIM and robot task planning: Creation and flow of construction-related information for action-level simulation of indoor wall frame installation, *Journal of Information Technology in Construction* 30 (2025) 352–374. doi:10.36680/j.itcon.2025.015.
- [4] A. Zhu, P. Pauwels, E. Torta, H. Zhang, B. De Vries, Data linking and interaction between BIM and robotic operating system (ROS) for flexible construction planning, *Automation in Construction* 163 (2024) 105426. doi:10.1016/j.autcon.2024.105426.
- [5] X. Wang, H. Yu, W. McGee, C. C. Menassa, V. R. Kamat, Enabling building information model-driven human-robot collaborative construction workflows with closed-loop digital twins,

Computers in Industry 161 (2024) 104112. doi:10.1016/j.compind.2024.104112.

[6] L. Xu, D. Veeramani, Z. Zhu, Automated physics-based modeling of construction equipment through data fusion, Automation in Construction 168 (2024) 105880. doi:10.1016/j.autcon.2024.105880.

[7] Alliance for OpenUSD, Openusd documentation, <https://openusd.org/release/>, accessed 26 July 2026 (2026).

[8] G. Ren, C. Huang, T. Su, From building deliverables to open scene description: A pipeline for lifecycle 3d interoperability, Buildings 15 (24) (2025) 4503. doi:10.3390/buildings15244503.

[9] NVIDIA Corporation, The simready standardization workflow, https://docs.omniverse.nvidia.com/simready/latest/simready-standards/standardization_workflow.html, accessed 26 July 2026 (2026).

[10] NVIDIA Corporation, Simready faq, <https://docs.omniverse.nvidia.com/simready/latest/simready-faq.html>, accessed 26 July 2026 (2026).

[11] X. Ye, L. Liu, M. König, Digital twin framework for humanoid robotics in construction: A conceptual approach for scene and task simulation, in: Proceedings of the 43rd International Symposium on Automation and Robotics in

Construction, 2026, pp. 438–445. doi:10.22260/ISARC2026/0057.

[12] P. Vähä, T. Heikkilä, P. Kilpeläinen, M. Järviluoma, E. Gambao, Extending automation of building construction: Survey on potential sensor technologies and robotic applications, Automation in Construction 36 (2013) 168–178. doi:10.1016/j.autcon.2013.08.002.

[13] T. Bock, The future of construction automation: Technological disruption and the upcoming ubiquity of robotics, Automation in Construction 59 (2015) 113–121. doi:10.1016/j.autcon.2015.07.022.

[14] H. Ardiny, S. Witwicki, F. Mondada, Construction automation with autonomous mobile robots: A review, in: 2015 3rd RSI International Conference on Robotics and Mechatronics, 2015, pp. 418–424. doi:10.1109/ICRoM.2015.7367821.

[15] J. M. Davila Delgado, L. Oyedele, A. Ajayi, L. Akanbi, O. Akinade, M. Bilal, H. Owolabi, Robotics and automated systems in construction: Understanding industry-specific challenges for adoption, Journal of Building Engineering 26 (2019) 100868. doi:10.1016/j.jobe.2019.100868.

[16] N. Melenbrink, J. Werfel, A. Menges, On-site autonomous construction robots: Towards unsupervised building, Automation in Construction 119 (2020)

103312. doi:10.1016/j.autcon.2020.103312.
- [17] B. Xiao, C. Chen, X. Yin, Recent advancements of robotics in construction, *Automation in Construction* 144 (2022) 104591. doi:10.1016/j.autcon.2022.104591.
- [18] K. H. Petersen, N. Napp, R. Stuart-Smith, D. Rus, M. Kovac, A review of collective robotic construction, *Science Robotics* 4 (28) (2019). doi:10.1126/scirobotics.aau8479.
- [19] Y. Xu, C. Cheung, M. S. Ng, A. Y. Kaltungo, T. Ishizawa, K. Fujimoto, Construction robotics: A systematic review of robot types, applications and human robot collaborations, *Results in Engineering* 30 (2026) 110406. doi:10.1016/j.rineng.2026.110406.
- [20] M. Afaq, S. Dutta, A. H. Gourabpasi, F. Jalaei, R. Ahmad, Robotics and AI for prefabrication of steel building components: Systematic review and future roadmap, *Automation in Construction* 190 (2026) 107107. doi:10.1016/j.autcon.2026.107107.
- [21] J. Zhang, H. Luo, J. Xu, Towards fully BIM-enabled building automation and robotics: A perspective of lifecycle information flow, *Computers in Industry* 135 (2022) 103570. doi:10.1016/j.compind.2021.103570.
- [22] W. Anane, I. Iordanova, C. Ouellet-Plamondon, Building information modeling (BIM) and robotic manufacturing technological interoperability in construction: A cyclic systematic literature review, *Digital Manufacturing Technology* (2023) 1–29doi:10.37256/dmt.3120231856.
- [23] H. Yue, Q. Wang, Z. Zhao, S. Lai, G. Huang, Interactions between BIM and robotics: Towards intelligent construction engineering and management, *Computers in Industry* 169 (2025) 104299. doi:10.1016/j.compind.2025.104299.
- [24] O. Odugu, F. Ghafari, E. Shourangiz, M. T. Khan, C. Wang, BIM-driven robotics in construction: A systematic review of integration methods, applications, and future directions, *Journal of Intelligent Construction* 3 (4) (2025) 1–17. doi:10.26599/JIC.2025.9180103.
- [25] M. Dawod, S. Hanna, BIM-assisted object recognition for the on-site autonomous robotic assembly of discrete structures, *Construction Robotics* 3 (1–4) (2019) 69–81. doi:10.1007/s41693-019-00021-9.
- [26] L. Ding, W. Jiang, Y. Zhou, C. Zhou, S. Liu, BIM-based task-level planning for robotic brick assembly through image-based 3d modeling, *Advanced Engineering Informatics* 43 (2020) 100993. doi:10.1016/j.aei.2019.100993.
- [27] O. Wong Chong, J. Zhang, R. M. Voyles, B.-C. Min, BIM-based simulation of con-

- struction robotics in the assembly process of wood frames, *Automation in Construction* 137 (2022) 104194. doi:10.1016/j.autcon.2022.104194.
- [28] Y. Gao, J. Meng, J. Shu, Y. Liu, BIM-based task and motion planning prototype for robotic assembly of COVID-19 hospitalisation light weight structures, *Automation in Construction* 140 (2022) 104370. doi:10.1016/j.autcon.2022.104370.
- [29] K. Kim, M. Peavy, BIM-based semantic building world modeling for robot task planning and execution in built environments, *Automation in Construction* 138 (2022) 104247. doi:10.1016/j.autcon.2022.104247.
- [30] M. S. Moura, C. Rizzo, D. Serrano, BIM-based localization and mapping for mobile robots in construction, in: 2021 IEEE International Conference on Autonomous Robot Systems and Competitions, 2021, pp. 12–18. doi:10.1109/ICARSC52212.2021.9429779.
- [31] M. A. Gopee, S. A. Prieto, B. García de Soto, Improving autonomous robotic navigation using IFC files, *Construction Robotics* 7 (3–4) (2023) 235–251. doi:10.1007/s41693-023-00112-8.
- [32] J. Chen, W. Lu, Y. Fu, Z. Dong, Automated facility inspection using robotics and BIM: A knowledge-driven approach, *Advanced Engineering Informatics* 55 (2023) 101838. doi:10.1016/j.aei.2022.101838.
- [33] X. Zhao, C. C. Cheah, A building information modeling (BIM)-enabled robotic system for automated indoor construction progress monitoring, in: 2024 IEEE International Conference on Real-time Computing and Robotics, 2024, pp. 38–43. doi:10.1109/RCAR61438.2024.10670745.
- [34] F. Bahreini, M. Nasrollahi, A. Taher, A. Hammad, Ontology for BIM-based robotic navigation and inspection tasks, *Buildings* 14 (8) (2024) 2274. doi:10.3390/buildings14082274.
- [35] M. Amani, R. Akhavian, Safe and trustworthy robot pathfinding with BIM, MHA*, and NLP, *Construction Robotics* 9 (2) (2025). doi:10.1007/s41693-025-00161-1.
- [36] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012, pp. 5026–5033. doi:10.1109/IRoS.2012.6386109.
- [37] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, Isaac gym: High performance GPU-based physics simulation for robot learning, *arXiv preprint arXiv:2108.10470* (2021). arXiv:2108.10470.
- [38] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain

randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2017, pp. 23–30. doi:10.1109/IRoS.2017.8202133.

[39] buildingSMART International, Industry foundation classes 4.3, https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/, accessed 26 July 2026 (2026).

[40] IfcOpenShell Contributors, Ifcopenshell 0.8.5 documentation, <https://docs.ifcopenshell.org/>, accessed 26 July 2026 (2026).

[41] C. Xia, K. Zhu, Z. Wang, F. Liu, Z. Zhang, Y. Duan, SimRecon: SimReady compositional scene reconstruction from real videos, arXiv preprint arXiv:2603.02133 (2026). arXiv:2603.02133. URL <https://arxiv.org/abs/2603.02133>

[42] X. Wang, L. Liu, T. Ding, A. Choi, C. Huang, M. Zhao, Z. Li, J. Jiang, C. Yu, S. Liu, W. Xu, Z. Su, Embodied Gen V2: An agentic, simulation-ready 3d world engine for embodied AI, arXiv preprint arXiv:2607.07459 (2026). arXiv:2607.07459. URL <https://arxiv.org/abs/2607.07459>

[43] G. Chen, Q. Xia, J. Peng, H. Zhang, B. Ma, J. Qian, Z. Jiao, B. Zhou,

L. Ye, K. Zhang, K. Wang, W. Zeng, Y. Chen, P. Yang, Z. Zeng, H. Wang, C. Liu, A. Yuille, F. Shi, C. Zheng, Y. Li, C. Jiang, P. Y. Chen, Agentic Real2Sim: Physics-based world modeling with vision-language agents, arXiv preprint arXiv:2607.19190 (2026). arXiv:2607.19190. URL <https://arxiv.org/abs/2607.19190>

[44] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, P. Abbeel, Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation, in: Robotics: Science and Systems XX, 2024, p. 61. doi:10.15607/RSS.2024.XX.061.

[45] C. Boje, A. Guerriero, S. Kubicki, Y. Rezgui, Towards a semantic construction digital twin: Directions for future research, Automation in Construction 114 (2020) 103179. doi:10.1016/j.autcon.2020.103179.

[46] R. Sacks, I. Brilakis, E. Pikas, H. S. Xie, M. Girolami, Construction with digital twin information systems, Data-Centric Engineering 1 (2020) e14. doi:10.1017/dce.2020.16.

[47] J. Tuomisto, Y. Zheng, I. Chauhan, O. Seppänen, J. Pajarinen, Automating on-site object inspection with a quadruped robot and BIM, Automation in Construction 181 (2026) 106571. doi:10.1016/j.autcon.2025.106571.

- [48] R. Akhavian, M. Amani, J. Mootz, B. Beheshti, R. Ashe, Semantic-gravity iterative closest point and AI-assisted navigation for robot-ready construction digital twins, *Automation in Construction* 190 (2026) 107134, article in press. doi:10.1016/j.autcon.2026.107134.
- [49] Q. Xu, A. Zhu, G. Xu, Z. Shao, J. Zhang, H. Zhang, FEM-based real-time task planning for robotic construction simulation, *Automation in Construction* 170 (2025) 105935. doi:10.1016/j.autcon.2024.105935.
- [50] A. Zhu, Z. Shao, X. Chai, E. Ma, Q. Li, H. Ye, H. Zhang, Component-based BIM-semantic web integration for enhanced robotic visual perception, *Automation in Construction* 177 (2025) 106270. doi:10.1016/j.autcon.2025.106270.
- [51] H. You, Y. Ye, T. Zhou, Q. Zhu, J. Du, Robot-enabled construction assembly with automated sequence planning based on ChatGPT: RoboGPT, *Buildings* 13 (7) (2023) 1772. doi:10.3390/buildings13071772.
- [52] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., Do as i can, not as i say: Grounding language in robotic affordances, in: *Conference on Robot Learning*, Vol. 205 of *Proceedings of Machine Learning Research*, 2023, pp. 287–318. URL <https://proceedings.mlr.press/v205/ahn23a.html>
- [53] W. Huang, F. Xia, D. Shah, D. Driess, A. Zeng, Y. Lu, P. Florence, I. Mordatch, S. Levine, K. Hausman, B. Ichter, Inner monologue: Embodied reasoning through planning with language models, in: *Conference on Robot Learning*, Vol. 205 of *Proceedings of Machine Learning Research*, 2023, pp. 1769–1782. URL <https://proceedings.mlr.press/v205/huang23c.html>
- [54] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, A. Zeng, Code as policies: Language model programs for embodied control, in: *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 9493–9500. doi:10.1109/ICRA48891.2023.10160591.
- [55] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, A. Garg, Progprompt: Generating situated robot task plans using large language models, in: *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 11523–11530. doi:10.1109/ICRA48891.2023.10161317.
- [56] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al., PaLM-E: An embodied multimodal language model, in: *Proceedings of the 40th International*

- Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 8469–8488.
 URL <https://proceedings.mlr.press/v202/driess23a.html> ²²³⁰
- [57] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al., RT-2: Vision-language-action models transfer web knowledge to robotic control, arXiv preprint arXiv:2307.15818 (2023). ²²⁰⁰ **arXiv:2307.15818.** ²²⁰⁵
- [58] S. S. Kannan, V. L. N. Venkatesh, B.-C. Min, SMART-LLM: Smart multi-agent robot task planning using large language models, arXiv preprint arXiv:2309.10062 (2023). **arXiv:2309.10062.** ²²¹⁰
- [59] X. Lin, Y. Wu, H. Yang, Y. Huang, Y. Zhang, J. Ji, Y. Zhang, Improving efficiency of answer set planning with rough solutions from large language models for robotic task planning, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, 2025, pp. 4570–4578. doi:10.24963/ijcai.2025/509. ²²¹⁵
- [60] M. Frasheri, G. Lumer-Klabbers, J. L. Kvistgaard, J. O. Hausted, C. Thule, L. Esterle, P. G. Larsen, Building safer robots: From simulation to hardware deployment, in: 2023 27th International Conference on Methods and Models in Automation and Robotics, 2023, pp. 63–68. doi:10.1109/MMAR58394.2023.10242467. ²²²⁰
- [61] buildingSMART International, Information delivery specification (ids) 1.0, <https://standards.buildingsmart.org/IDS/>, accessed 13 September 2026 (2024).
- [62] buildingSMART International, buildingsmart data dictionary (bsdd), <https://www.buildingsmart.org/users/services/buildingsmart-data-dictionary/>, accessed 13 September 2026 (2025). ²²²⁵